

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерного проектирования
Кафедра инженерной психологии и эргономики
Дисциплина: Компьютерные системы и сети

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему

**ВЕБ-ПРИЛОЖЕНИЕ, РЕАЛИЗУЮЩЕЕ СЕТЕВУЮ ИГРУ «MEMORY:
НАЙДИ ПАРУ»**

БГУИР КП 6-05-0612-01 001ПЗ

Студент

Кавцевич В. А.

Руководитель

Болтак С. В.

Минск 2026

СОДЕРЖАНИЕ

Введение.....	5
1 Анализ предметной области	7
1.1 Обзор аналогов	7
1.2 Постановка задачи	11
2 Проектирование программного средства	13
2.1 Архитектура приложения.....	13
2.2 Проектирование сетевого взаимодействия	13
2.3 Проектирование игрового процесса.....	14
2.4 Проектирование пользовательского интерфейса	15
3 Разработка программного средства.....	17
3.1 Реализация сетевой части.....	17
3.2 Реализация игровой логики.....	18
3.3 Разработка интерфейса	19
3.4 Дополнительные функции	23
4 Тестирование	25
5 Руководство пользователя.....	33
5.1 Описание пользовательского интерфейса	33
5.2 Управление игровым процессом	36
5.3 Правила игры.....	37
Заключение	38
Список используемых источников.....	40
Приложение А	41

ВВЕДЕНИЕ

В современном мире цифровые технологии оказывают значительное влияние на трансформацию традиционных форм досуга. Настольные игры, десятилетиями остававшиеся атрибутом семейного вечера или дружеской встречи, сегодня активно мигрируют в виртуальное пространство. Этот процесс не только расширяет аудиторию, но и открывает новые горизонты для взаимодействия между игроками, позволяя преодолевать географические барьеры. Одной из таких игр, получившей широкое признание во всем мире, является «Memory: Найди пару». Простота правил, минимальные требования к оборудованию и доказанная польза для развития когнитивных способностей делают эту игру идеальным кандидатом для переноса в цифровой формат с поддержкой многопользовательского режима.

Актуальность данного курсового проекта определяется тремя ключевыми аспектами. Наблюдается устойчивый рост интереса к браузерным играм, не требующим установки дополнительного программного обеспечения и доступным на различных устройствах. Существующие веб-реализации игры «Memory» преимущественно ориентированы на одиночный режим или игру на одном устройстве, что существенно ограничивает их соревновательный потенциал. В то же время современные веб-технологии, такие как ASP.NET Core и встроенная поддержка WebSocket, позволяют создавать полноценные многопользовательские приложения с синхронизацией в реальном времени, что открывает возможность для разработки качественного игрового продукта, сочетающего классическую механику с современными сетевыми возможностями.

Целью данного курсового проекта является разработка веб-приложения, реализующего сетевую версию игры «Memory: Найди пару» с поддержкой многопользовательского режима и синхронизацией игрового состояния в реальном времени.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1 Провести анализ предметной области, включающий изучение существующих аналогов, выявление их преимуществ и недостатков, а также формулирование требований к разрабатываемой программе.
- 2 Спроектировать архитектуру приложения, определить структуру клиентской и серверной частей, разработать схему сетевого взаимодействия и игровой логики.
- 3 Реализовать серверную часть приложения на платформе ASP.NET Core с использованием собственного WebSocket-сервера для обеспечения двусторонней связи в реальном времени.
- 4 Разработать клиентскую часть приложения на HTML, CSS и JavaScript с адаптивным дизайном, обеспечивающим корректное отображение на различных устройствах.

5 Реализовать игровую логику, соответствующую классическим правилам игры «Memory: Найди пару».

6 Провести тестирование разработанного приложения для выявления и устранения ошибок.

7 Разработать руководство пользователя, описывающее интерфейс и правила игры.

В результате выполнения работы создано веб-приложение, позволяющее игрокам создавать игровые комнаты, присоединяться к ним и соревноваться в поиске пар карточек через браузер. В результате разработано приложение, которое поддерживает ключевые аспекты игры: формирование игрового поля, отслеживание ходов, синхронизацию состояния игры между всеми участниками в реальном времени.

Данная пояснительная записка описывает процесс разработки программного средства, включая анализ, проектирование, реализацию, тестирование и инструкции для пользователей. Работа направлена на демонстрацию возможностей современных веб-технологий в создании интерактивных многопользовательских приложений.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор аналогов

В рамках анализа предметной области были рассмотрены существующие веб-реализации игры «Memory: Найди пару»: PairsOne.ru и mozgotren.com trenagors memory. Цель анализа – изучить их функциональные возможности, особенности интерфейса и уровень поддержки многопользовательского режима, а также выявить преимущества и недостатки для формирования требований к разрабатываемому программному средству.

Pairs One – это браузерная платформа для игры Memory, предоставляющая пользователям возможность играть как в одиночном, так и в многопользовательском режиме. Сервис позволяет создавать игровые сессии и делиться ссылкой с другими игроками для совместной игры в режиме реального времени. Пользователи могут выбирать различные режимы, включая игру со случайным соперником или локальную игру на одном устройстве. Интерфейс (рисунки 1.1, 1.2) приложения минималистичен и ориентирован на быстрый запуск игры без необходимости регистрации [2].

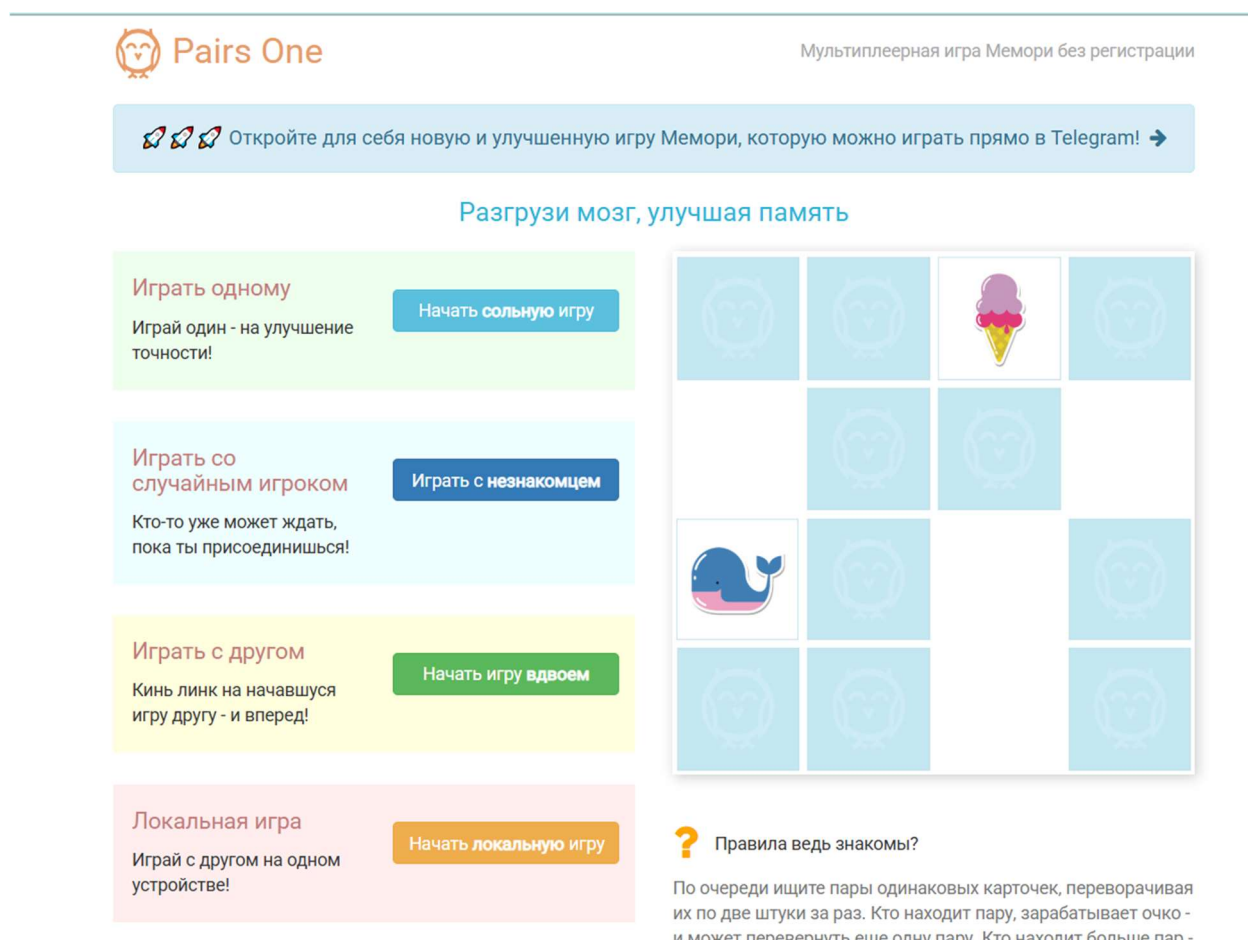


Рисунок 1.1 – Интерфейс Pairs One

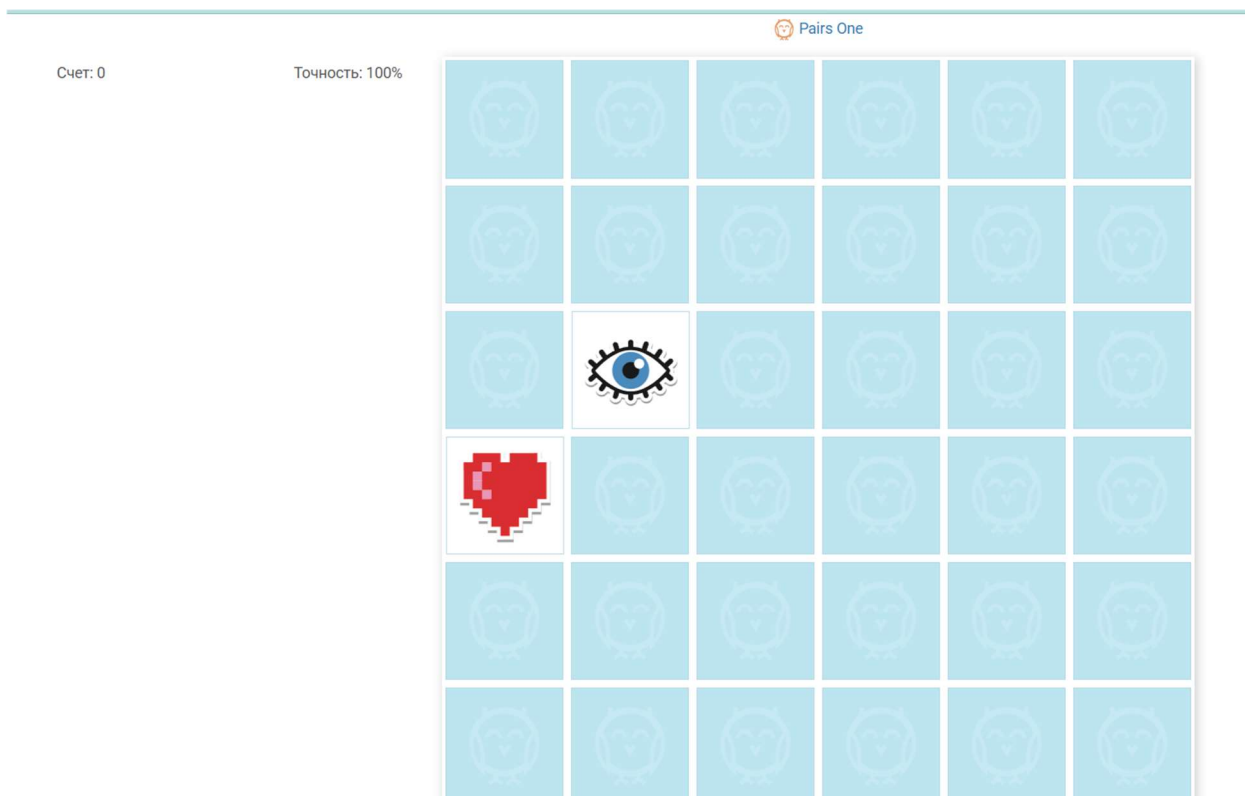


Рисунок 1.2 – Интерфейс Pairs One

Основные функции:

- реализация классической механики игры с открытием карточек и поиском совпадающих пар;
- поддержка различных режимов игры: одиночный, многопользовательский, локальный;
- создание игровых комнат с возможностью подключения по ссылке;
- подсчёт очков и определение победителя.

Платформы: браузер (ПК и мобильные устройства), не требует установки.

Преимущества:

- поддержка сетевой игры в реальном времени;
- простота подключения к игровой сессии;
- отсутствие необходимости регистрации;
- быстрый запуск игры.

Недостатки:

- минималистичный интерфейс с ограниченной визуальной составляющей;
- нет дополнительных функций (чат, история игр).

Mozgotren – это веб-платформа, предоставляющая тренажёры для развития когнитивных способностей, включая игру Memory. Сервис ориентирован на тренировку памяти и внимания, предлагая пользователям

различные уровни сложности и систему оценки результатов с учётом правильности действий и скорости выполнения. Помимо одиночного режима, платформа поддерживает соревновательный формат, позволяющий пользователям сравнивать результаты. Интерфейс (рисунки 1.3, 1.4) содержит элементы статистики и обучения, что отличает его от классических игровых реализаций [3].

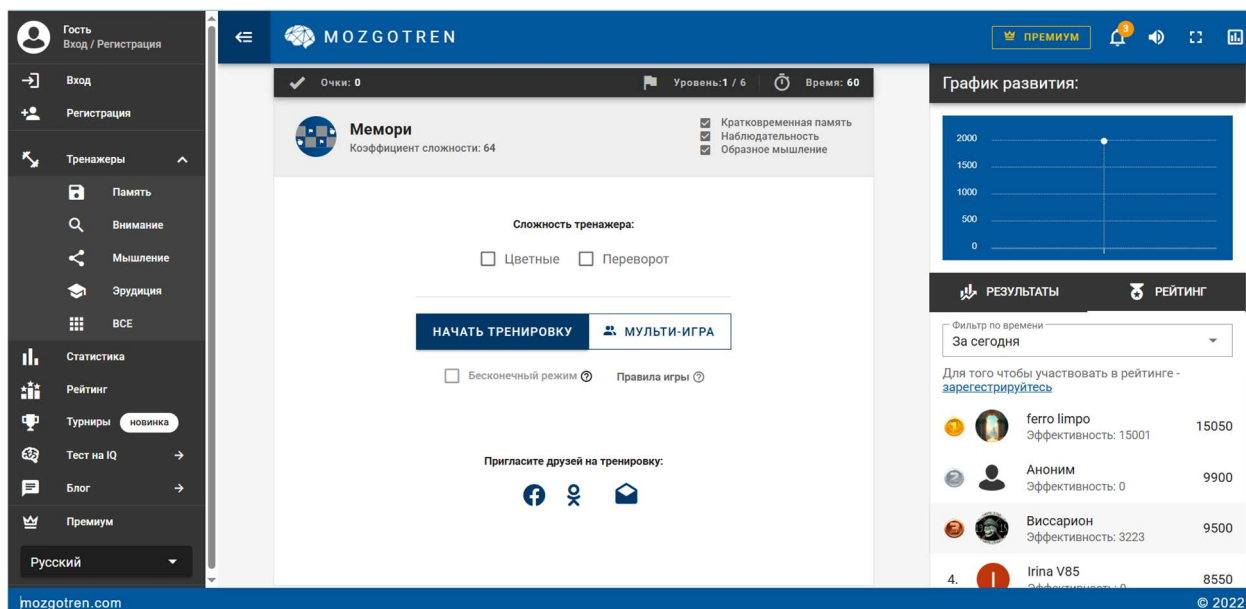


Рисунок 1.3 – Интерфейс Mozgotren

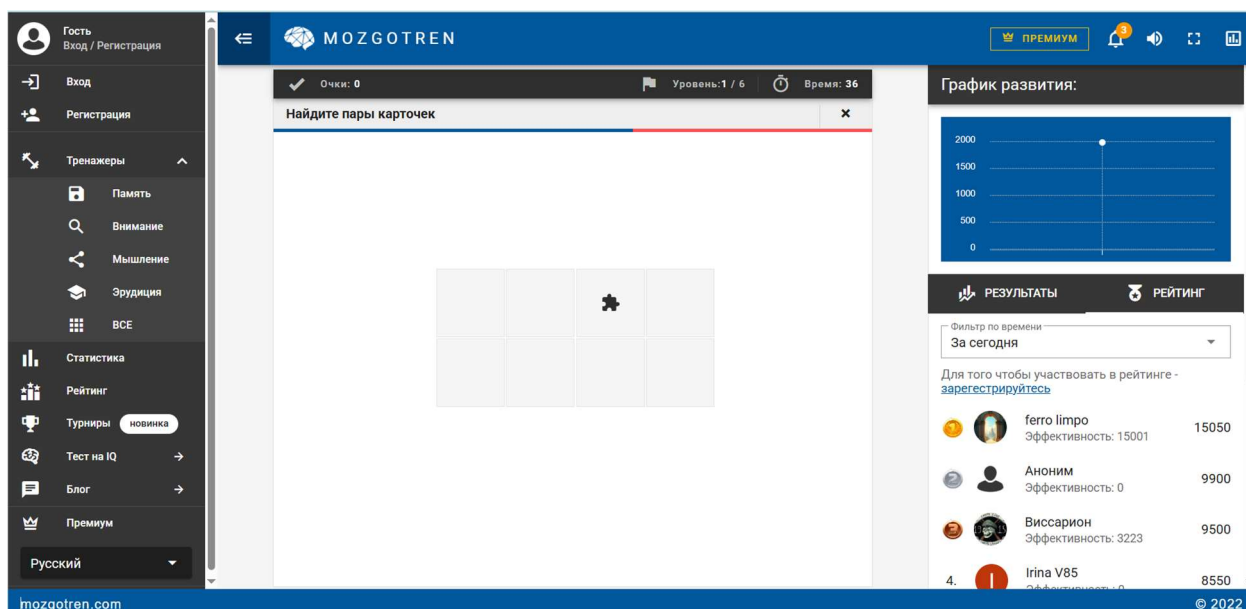


Рисунок 1.4 – Интерфейс Mozgotren

Основные функции:

- реализация игры «Memory» как тренажёра для развития когнитивных способностей;
- настройка сложности и использование коэффициентов сложности;
- подсчёт очков с учётом уровня и правильности ответов;
- многопользовательский режим с соревнованием между игроками в реальном времени;
- система штрафов за ошибки и бонусов за правильные действия.

Платформы: браузер (ПК и мобильные устройства).

Преимущества:

- расширенная система оценки результатов (баллы, уровень, коэффициенты);
- развитие различных когнитивных навыков: памяти, внимания, логического мышления;
- наличие соревновательного режима в реальном времени.

Недостатки:

- сложность интерфейса по сравнению с классической игрой;
- ориентация на тренировку, а не на игровой процесс;
- ограниченная интуитивность для новых пользователей.

Сравнение с разрабатываемым программным средством:

Разработанное веб-приложение, реализующее игру «Memory: Найди пару» с использованием ASP.NET Core и собственного WebSocket-сервера, обладает рядом преимуществ по сравнению с рассмотренными аналогами:

- сетевая синхронизация: использование WebSocket-соединения обеспечивает мгновенное обновление игрового состояния для всех участников;
- баланс между игрой и обучением: в отличие от Mozgotren, приложение ориентировано именно на игровой процесс, а не на тренировку;
- удобство использования: более простой и интуитивный интерфейс по сравнению с обучающими платформами;
- функциональность: возможность расширения (статистика игр);
- доступность: отсутствие необходимости установки и регистраций.

Таким образом, анализ показал, что существующие решения либо ориентированы на простой быстрый запуск игры с минимальным интерфейсом, либо на тренировку когнитивных способностей с избыточной системой оценки и сложностью интерфейса, но не обеспечивают полноценного сочетания удобного и визуально привлекательного интерфейса со стабильным многопользовательским взаимодействием в реальном времени. Это подтверждает актуальность разработки собственной программы, позволяющей реализовать классическую игровую механику с надёжной сетевой синхронизацией.

1.2 Постановка задачи

На основе проведённого анализа предметной области и рассмотренных аналогов сформулирована задача разработки веб-приложения, реализующего многопользовательскую версию игры «Memory: Найди пару».

Цель разработки заключается в создании удобного и функционального программного решения, обеспечивающего возможность игры в режиме реального времени через веб-браузер с сохранением классических правил и обеспечением надёжного сетевого взаимодействия между пользователями.

Для достижения поставленной цели необходимо решить следующие функциональные задачи:

- организация игровых сессий: обеспечить возможность создания игровых комнат и подключения пользователей к существующим сессиям по уникальному идентификатору или ссылке;
- генерация игрового поля: реализовать формирование игрового поля заданного размера с карточками, случайным образом объединёнными в пары;
- реализация игровой механики: обеспечить обработку действий пользователей, включая открытие карточек, проверку совпадений и возврат карточек в исходное состояние при несовпадении;
- поддержка многопользовательского режима: реализовать взаимодействие нескольких игроков в рамках одной игровой сессии с поочерёдным выполнением ходов;
- обеспечение синхронизации: гарантировать обновление игрового состояния у всех участников в режиме реального времени при любых изменениях;
- подсчёт результатов игры: реализовать механизм определения победителя на основе количества найденных пар или набранных очков;
- разработка пользовательского интерфейса: создать удобный и интуитивно понятный интерфейс, включающий основные элементы управления игровым процессом.

Помимо функциональных возможностей, разрабатываемое приложение должно соответствовать ряду нефункциональных требований:

- кроссплатформенность: обеспечить корректную работу в популярных веб-браузерах на различных устройствах без необходимости установки дополнительного программного обеспечения;
- производительность: обеспечить минимальное время отклика системы и быструю загрузку элементов интерфейса;
- надёжность: гарантировать стабильную работу приложения и корректную обработку возможных ошибок, включая разрыв соединения;
- масштабируемость: предусмотреть возможность одновременного участия нескольких пользователей без существенного ухудшения качества работы системы;
- удобство использования: обеспечить простоту освоения интерфейса и комфортное взаимодействие пользователя с приложением.

Для реализации поставленных задач предполагается использование следующих технологий:

- серверная часть: ASP.NET Core, обеспечивающая обработку запросов и управление логикой приложения [4];
- сетевое взаимодействие: технология WebSocket-сервер на встроенных средствах .NET, для организации двустороннего обмена данными и синхронизации в реальном времени [5];
- клиентская часть: HTML, CSS и JavaScript для создания адаптивного пользовательского интерфейса [6].

Разрабатываемое программное средство ориентировано на создание современного веб-приложения, сочетающего простоту классической игры «Memory: Найди пару» с возможностями многопользовательского взаимодействия и актуальными веб-технологиями.

2 ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

2.1 Архитектура приложения

Приложение планируется построить по клиент-серверной архитектуре. Серверная часть будет отвечать за обработку подключений, управление игровыми комнатами, выполнение игровой логики и синхронизацию состояния между игроками. Клиентская часть будет представлять собой веб-интерфейс, работающий в браузере, и будет отвечать за отображение игрового процесса и передачу действий пользователя на сервер.

Серверная часть должна включать следующие компоненты:

- модуль для приёма и обработки WebSocket-соединений;
- менеджер игровых комнат для создания, хранения и удаления игровых сессий;
- модель игровой комнаты, содержащую список игроков, игровое поле и состояние игры;
- контроллер для отображения веб-страниц и раздачи статических файлов.

Клиентская часть должна включать:

- HTML-шаблон с тремя экранами (лобби, ожидание, игра);
- стили для адаптивного отображения и анимаций;
- JavaScript-модуль для установки WebSocket-соединения, отправки команд и обработки входящих сообщений.

Вся игровая логика должна выполняться на сервере. Клиент должен только отображать полученное состояние и передавать действия пользователя. Такой подход обеспечит синхронизацию игры между всеми участниками и предотвратит возможность читерства.

2.2 Проектирование сетевого взаимодействия

Для обеспечения двусторонней связи между клиентом и сервером в реальном времени планируется использовать протокол WebSocket. Данный протокол позволяет установить постоянное соединение, по которому сервер может отправлять данные клиенту без необходимости повторных HTTP-запросов.

Процесс установки соединения должен включать следующие этапы:

- 1 Клиент отправляет HTTP-запрос с заголовками Upgrade: websocket и Connection: Upgrade, содержащий уникальный ключ.
- 2 Сервер вычисляет ответный ключ на основе полученного и отправляет ответ с кодом 101 Switching Protocols.
- 3 После успешного рукопожатия TCP-соединение не закрывается, а переходит в режим постоянного двустороннего обмена сообщениями.

Все сообщения между клиентом и сервером планируется передавать в формате JSON, что обеспечит удобство отладки и гибкость при расширении функционала.

Предусматриваются следующие типы команд от клиента к серверу:

- создание новой игровой комнаты с указанным размером поля;
- подключение к существующей комнате по уникальному коду;
- начало игры (доступно только создателю комнаты);
- открытие карточки на игровом поле;
- подтверждение перехода хода после несовпадения карт;
- принудительное обновление состояния игры при повторном подключении;
- поддержание соединения (ping-сообщения).

Сервер, в свою очередь, должен отправлять клиентам следующие типы сообщений:

- подтверждение создания комнаты с её кодом;
- обновлённый список игроков в комнате;
- начало игры с передачей состояния игрового поля;
- уведомление об открытии карты;
- сообщение о найденной паре с обновлением счёта;
- уведомление о несовпадении карт;
- смена текущего игрока;
- завершение игры с указанием победителя.

Для поддержки нескольких игроков одновременно сервер должен использовать асинхронную обработку подключений. Каждое входящее соединение должно обрабатываться в отдельной задаче, что позволит серверу обслуживать множество клиентов без блокировки основного потока.

Для обеспечения стабильности соединения клиент должен отправлять периодические ping-сообщения. При разрыве соединения клиент должен предпринимать несколько попыток переподключения с увеличивающейся задержкой.

2.3 Проектирование игрового процесса

Игровой процесс проектируется в соответствии с классическими правилами игры «Memory: Найди пару» и должен включать следующие этапы.

Подготовка к игре: Игрок, создающий комнату, должен иметь возможность выбирать размер игрового поля (4×4, 6×6 или 8×8). После подключения не менее двух игроков создатель комнаты нажимает кнопку «Начать игру». Сервер должен сгенерировать игровое поле: создать пары карточек (количество пар равно половине от общего числа карт), каждой паре присвоить уникальный идентификатор и путь к изображению, после чего все карточки случайным образом перемешать.

Механика хода: Игроки должны ходить по очереди. В свой ход игрок может открыть две карточки:

- при открытии первой карточки она переворачивается лицевой стороной вверх, сервер запоминает её индекс;

- при открытии второй карточки сервер проверяет совпадение изображений. Если изображения совпадают, обе карточки остаются открытыми, помечаются как найденные, игрок получает одно очко, и его ход продолжается. Если изображения не совпадают, карточки должны перевернуться обратно через короткую задержку, после чего ход переходит к следующему игроку.

Блокировка интерфейса: для предотвращения некорректных действий во время анимации переворота карточек должна быть реализована система блокировки интерфейса. Пока выполняется анимация, нажатия на карточки должны игнорироваться. Также блокировка должна действовать, когда ходит другой игрок.

Подсчёт очков и определение победителя: каждая найденная пара должна приносить игроку одно очко. Игра завершается, когда все пары карточек найдены. Победителем должен объявляться игрок, набравший наибольшее количество очков. При равенстве очков должна объявляться ничья.

Завершение игры и перезапуск: после окончания игры всем участникам должно отображаться окно с результатами. Создатель комнаты должен иметь возможность начать новую игру без повторного подключения. При перезапуске сервер должен сбросить состояние комнаты (обнулить счёт игроков, сгенерировать новое поле) и начать игру заново.

2.4 Проектирование пользовательского интерфейса

Пользовательский интерфейс планируется выполнить в виде одностраничного веб-приложения с динамической сменой экранов. Все изменения состояния должны отображаться без перезагрузки страницы для обеспечения плавности и отзывчивости интерфейса.

Интерфейс должен включать три основных экрана: экран лобби, экран ожидания и игровой экран.

Экран лобби должен содержать две формы:

- форма «Создать комнату» с полем для ввода имени игрока, выпадающим списком для выбора размера поля и кнопкой создания;

- форма «Присоединиться» с полем для ввода имени игрока, полем для ввода кода комнаты и кнопкой подключения.

Обе формы должны иметь скруглённые углы, визуальное увеличение при наведении и подсветку полей ввода при фокусе.

Экран ожидания должен отображаться после создания комнаты или успешного подключения. Он должен содержать:

- код комнаты, отображаемый крупным шрифтом для удобства передачи другим игрокам;

- выбранный размер поля;

- список подключившихся игроков (текущий игрок должен быть помечен);
- кнопку «Начать игру» (только для создателя комнаты);
- кнопку «Выйти из комнаты»;
- информационный текст об ожидании подключения других игроков.

Игровой экран должен отображаться после начала игры. Он должен содержать:

- верхнюю панель с кодом комнаты и индикатором текущего хода (индикатор должен подсвечиваться, когда ходит текущий игрок);
- панель игроков с именами участников и их счетами (текущий игрок должен визуально выделяться);
- игровое поле в виде сетки карточек (размер сетки зависит от выбранного размера поля);
- кнопку «Выйти из игры».

Карточки в закрытом состоянии должны отображаться в виде квадрата с вопросительным знаком. При открытии на карточке должно появляться изображение. Найденные пары должны отмечаться зелёным фоном и становиться недоступными для нажатия.

Модальное окно окончания игры должно отображаться после завершения партии. Окно должно содержать:

- иконку кубка;
- сообщение о победителе (или ничье);
- кнопку «Сыграть ещё раз» (только для создателя комнаты) и кнопку «Закрыть».

Интерфейс должен быть адаптивным и корректно отображаться на устройствах с разной шириной экрана. На мобильных устройствах формы должны выстраиваться вертикально, размер карточек и отступы должны автоматически уменьшаться.

3 РАЗРАБОТКА ПРОГРАММНОГО СРЕДСТВА

3.1 Реализация сетевой части

Сетевая часть приложения реализована на основе протокола WebSocket, работающего поверх TCP. Для обеспечения двусторонней связи между клиентом и сервером в реальном времени разработан собственный WebSocket-сервер RawWebSocketHandler, реализующий протокол RFC 6455 вручную [7]. Данное решение позволяет избежать использования сторонних библиотек и даёт полный контроль над форматом передаваемых сообщений.

Сервер запускается на порту 8081. При старте создаётся TCP-слушатель, который принимает входящие подключения. Для каждого подключившегося клиента создаётся отдельная асинхронная задача, что позволяет серверу обрабатывать несколько соединений одновременно без блокировки основного потока.

Процесс установки WebSocket-соединения начинается с того, что клиент отправляет HTTP-запрос с заголовками Upgrade: websocket и Connection: Upgrade, содержащий ключ Sec-WebSocket-Key. Сервер извлекает этот ключ, добавляет к нему специальную строку-константу, вычисляет хеш SHA-1 и кодирует результат в Base64. После этого сервер отправляет ответ с кодом 101 Switching Protocols и вычисленным ключом. После успешного рукопожатия TCP-соединение переходит в режим постоянного двустороннего обмена сообщениями.

Все игровые сообщения между клиентом и сервером передаются в формате JSON. Сервер обрабатывает следующие типы команд от клиента: create_room (создание комнаты с указанным размером поля), join_room (подключение к существующей комнате по коду), start_game (начало игры, доступно только создателю комнаты), restart_game (перезапуск игры), flip_card (открытие карточки), next_turn (подтверждение перехода хода после несовпадения), force_update (принудительное обновление состояния) и ping (поддержание соединения).

Сервер, в свою очередь, отправляет клиентам следующие типы сообщений: room_created (подтверждение создания комнаты с её кодом), players_updated (обновлённый список игроков), game_started (начало игры с передачей состояния поля), card_flipped (уведомление об открытии карты), match_found (сообщение о найденной паре), no_match (уведомление о несовпадении), turn_changed (смена текущего игрока), game_over (завершение игры), game_restarted (перезапуск игры), player_left (уведомление о выходе игрока) и error (сообщение об ошибке).

Для синхронизации состояния все изменения после обработки на сервере рассылаются всем клиентам в комнате. Для потокобезопасной отправки сообщений используется механизм SemaphoreSlim, который предотвращает одновременную отправку нескольких сообщений одному клиенту из разных потоков.

При разрыве соединения сервер удаляет игрока из комнаты, уведомляет остальных участников с помощью сообщения `player_left` и обновляет список игроков. Если комната становится пустой, она удаляется из менеджера комнат. Для обнаружения разрывов соединения клиент отправляет `ping`-сообщения каждые 30 секунд. При разрыве соединения клиент предпринимает до пяти попыток переподключения с увеличивающейся задержкой.

3.2 Реализация игровой логики

Игровая логика реализована на сервере в классах `GameRoomManager` и `GameRoom`. Клиент только отображает текущее состояние и передаёт действия пользователя (авторитарный подход), что обеспечивает синхронизацию игры между всеми участниками и предотвращает читерство.

Класс `GameRoomManager` хранит все активные комнаты в потокобезопасной коллекции `ConcurrentDictionary`. При создании комнаты генерируется уникальный 4-значный код, состоящий из букв латинского алфавита и цифр. Комнаты могут быть созданы с размером поля 4×4 , 6×6 или 8×8 в зависимости от выбора пользователя. Максимальное количество игроков в одной комнате ограничено четырьмя.

Генерация игрового поля выполняется с помощью метода `InitializeBoard`. Количество карточек равно квадрату размера поля (16 для 4×4 , 36 для 6×6 , 64 для 8×8). Алгоритм создания поля: сначала формируются пары карточек с одинаковым идентификатором пары и путём к изображению, после чего все карточки случайным образом перемешиваются с использованием алгоритма Фишера-Йетса [8].

При получении команды `flip_card` сервер последовательно проверяет: существует ли комната, начата ли игра и не закончена ли она, не выполняется ли обработка другого хода, соответствует ли игрок текущему ходу, не открыта ли уже карточка и не найдена ли она ранее. Если открывается первая карта в ходу, она переворачивается, сервер запоминает её индекс и рассылает уведомление всем игрокам. Если открывается вторая карта, сервер проверяет совпадение пар. При совпадении обе карточки помечаются как найденные, счёт игрока увеличивается, и рассылается сообщение о найденной паре. При несовпадении устанавливается флаг обработки, рассылается уведомление о несовпадении, после чего ожидается команда `next_turn` для перехода хода.

При получении команды `next_turn` сервер переворачивает открытые карты обратно, очищает индекс первой открытой карты, переключает текущего игрока на следующего и сбрасывает флаг обработки. После этого всем игрокам рассылается сообщение о смене хода с идентификатором следующего игрока.

Игра завершается, когда все карточки на поле помечены как найденные. Победителем объявляется игрок, набравший наибольшее количество очков. Если несколько игроков набрали одинаковое количество очков, объявляется

ничья. После завершения игры всем игрокам рассылается сообщение `game_over`.

Создатель комнаты может отправить команду `restart_game`. Сервер сбрасывает состояние комнаты: обнуляет счёт всех игроков, генерирует новое поле, сбрасывает флаги состояния игры. После этого всем игрокам рассылается сообщение `game_restarted`, и игроки возвращаются на экран ожидания.

3.3 Разработка интерфейса

Клиентская часть разработана как одностраничное веб-приложение на HTML, CSS и JavaScript без использования дополнительных фреймворков [9].

Основной HTML-файл содержит три экрана: экран лобби (создание или подключение к комнате), экран ожидания игроков и игровой экран. Переключение между экранами осуществляется сменой CSS-класса `active`. Все экраны изначально имеют стиль `display: none`, а активный экран получает `display: block` с анимацией появления.

Экран лобби содержит две формы. Форма «Создать комнату» включает поле для ввода имени игрока, выпадающий список для выбора размера поля (4×4 — лёгкий уровень, 6×6 — средний уровень, 8×8 — сложный уровень) и кнопку «Создать игру». Форма «Присоединиться» включает поле для ввода имени игрока, поле для ввода кода комнаты (4 символа, автоматическое преобразование в верхний регистр) и кнопку «Присоединиться». На рисунке 3.1 представлен экран лобби.

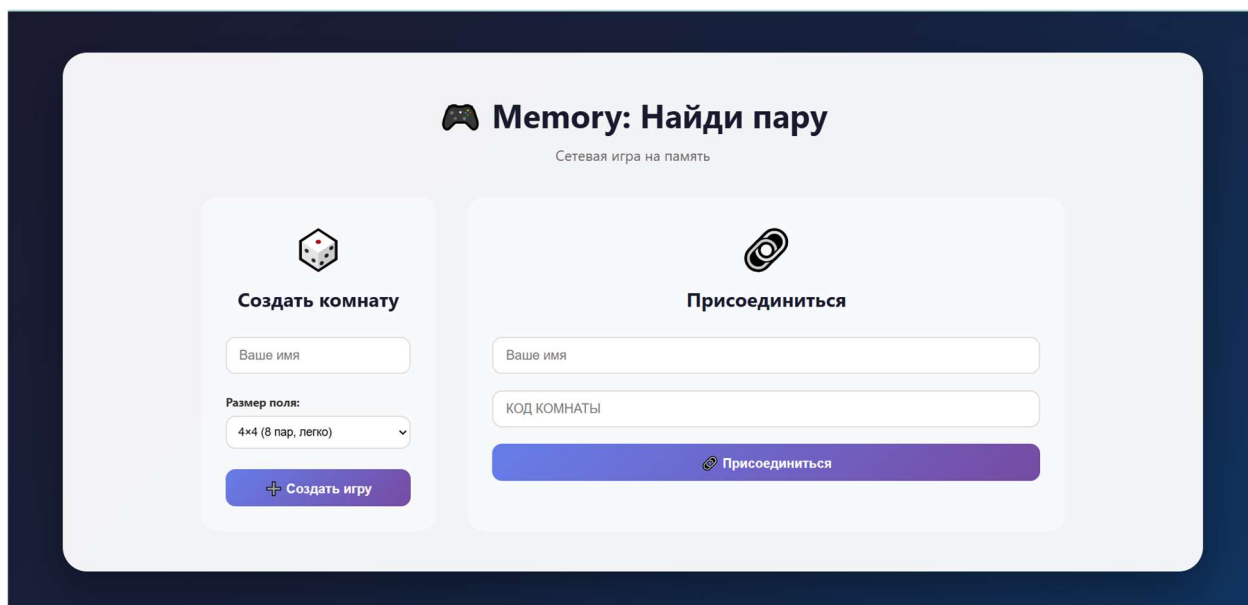


Рисунок 3.1 – Экран лобби

Экран ожидания отображается после создания комнаты или успешного подключения. На нём показан код комнаты крупным шрифтом для удобства

передачи другим игрокам, выбранный размер поля, список подключившихся игроков (текущий игрок помечен подписью «(Вы)»), кнопка «Начать игру» (отображается только для создателя комнаты), кнопка «Выйти из комнаты» и информационный текст об ожидании подключения других игроков. Список игроков обновляется автоматически через WebSocket-сообщения типа `players_updated`. На рисунке 3.2 представлен экран ожидания.

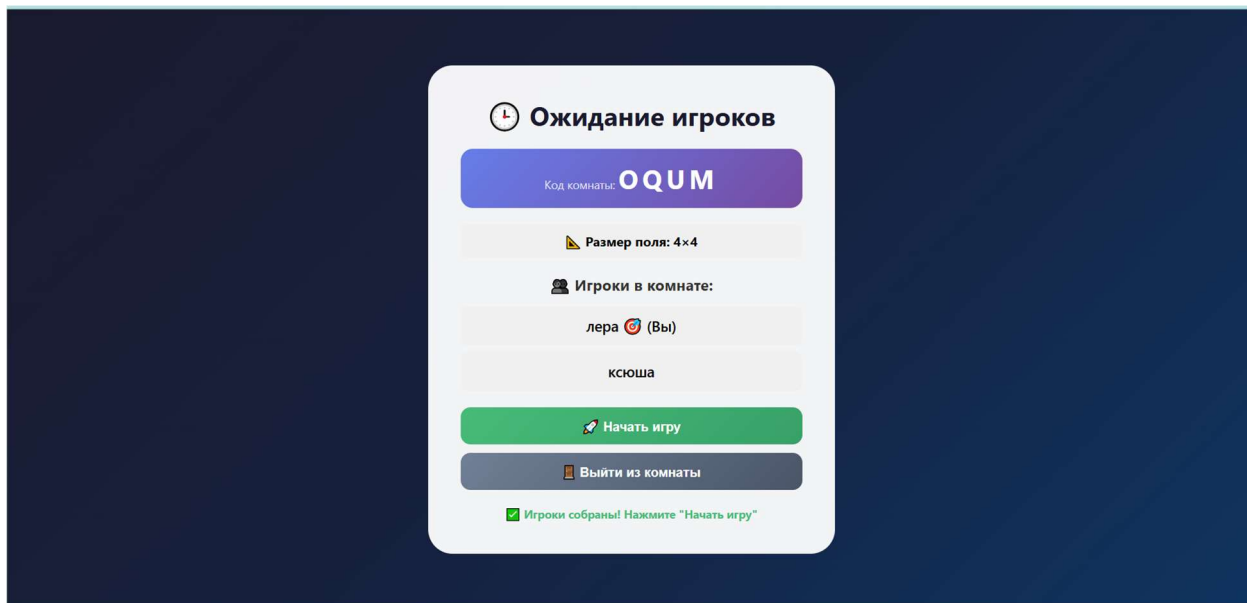


Рисунок 3.2 – Экран ожидания

Игровой экран отображается после нажатия кнопки «Начать игру». Он содержит верхнюю панель с кодом комнаты и индикатором текущего хода (индикатор подсвечивается зелёным цветом и пульсирует, когда ходит текущий игрок), панель игроков с именами участников и их счетами (текущий игрок визуально выделяется), игровое поле в виде сетки карточек и кнопку «Выйти из игры». Размер сетки зависит от выбранного размера поля: 4×4, 6×6 или 8×8. Поле автоматически центрируется с помощью CSS Grid и масштабируется под размер окна браузера. На рисунке 3.3 представлен игровой экран.

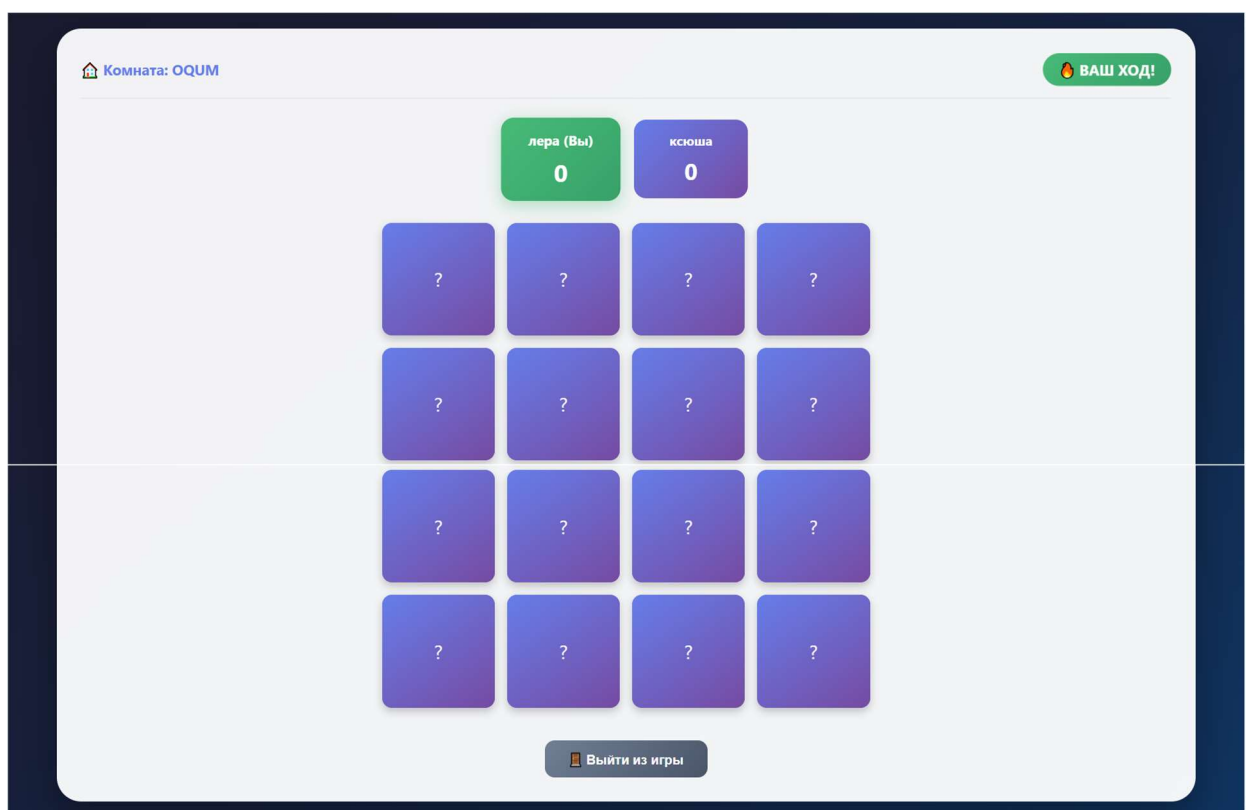


Рисунок 3.3 – Игровой экран

Карточки в закрытом состоянии отображаются в виде фиолетово-синего градиентного квадрата с вопросительным знаком в центре. При открытии карточки на ней появляется соответствующее изображение. Найденные пары карточек отмечаются зелёным фоном и символом «✓», после чего становятся недоступными для нажатия. На рисунке 3.4 представлен игровой экран с найденными парами.

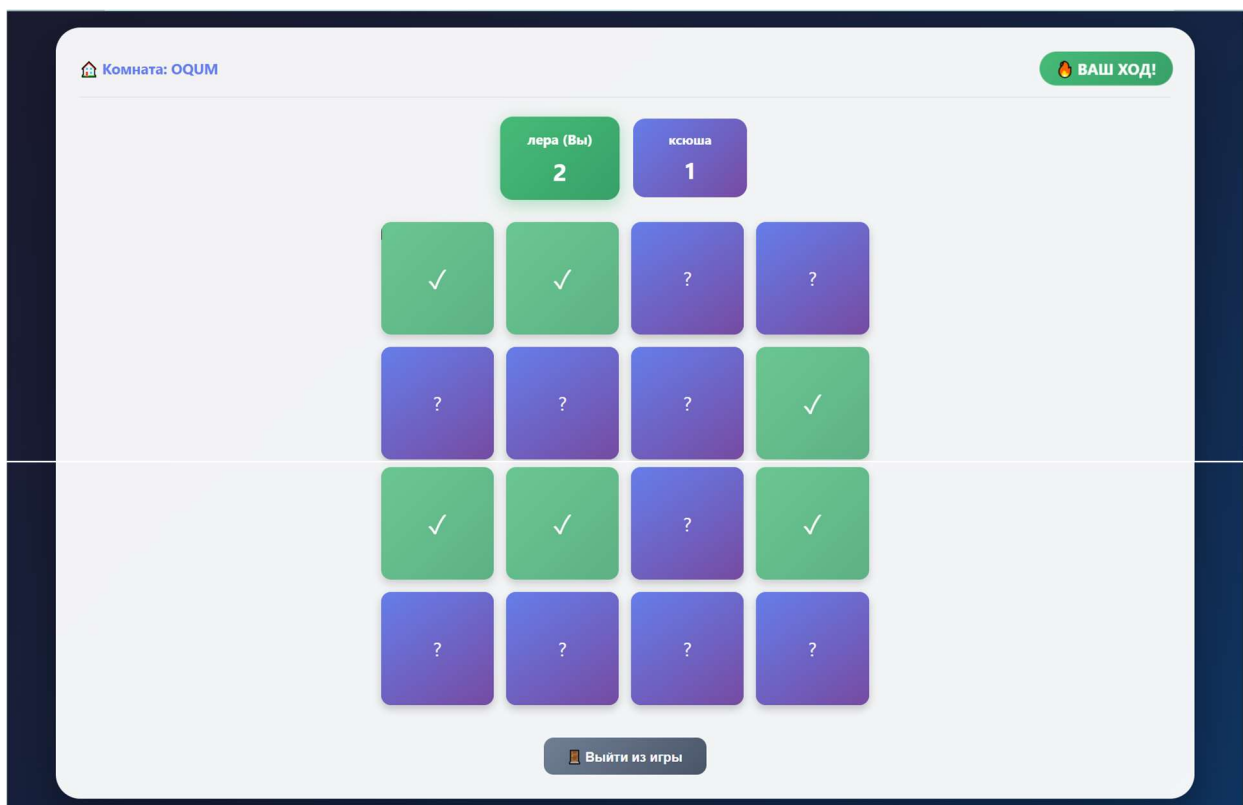


Рисунок 3.4 – Игровой экран с найденными парами

При завершении игры поверх игрового экрана отображается модальное окно с полупрозрачным тёмным фоном. Окно содержит иконку кубка, сообщение о победителе (при победе текущего игрока выводится текст «Поздравляем! Вы победили!», при победе соперника — «Победил: [имя игрока]», при ничьей — «Ничья!»), а также две кнопки: «Сыграть ещё раз» (доступна только создателю комнаты) и «Закрыть». При нажатии на «Сыграть ещё раз» отправляется команда `restart_game`, после чего сервер сбрасывает состояние комнаты и игра начинается заново. На рисунке 3.5 представлено модальное окно.

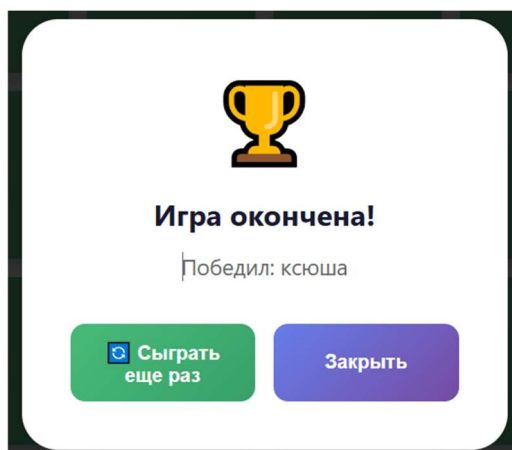


Рисунок 3.5 – Модальное окно

Для стилизации интерфейса используется файл CSS, в котором реализованы: сетка игрового поля с помощью CSS Grid, единое соотношение сторон карточек 1:1 с помощью свойства aspect-ratio, анимации плавного появления экранов и пульсации индикатора хода. Интерфейс адаптирован для мобильных устройств с помощью медиа-запросов: при ширине экрана менее 768 пикселей формы на экране лобби выстраиваются вертикально, а размер карточек и отступы уменьшаются.

Клиентская логика реализована в файле JavaScript. При загрузке страницы генерируется или загружается из localStorage уникальный идентификатор подключения, устанавливается WebSocket-соединение с сервером, настраиваются обработчики событий WebSocket и запускается интервал для отправки ping-сообщений каждые 30 секунд. Функция handleMessage обрабатывает входящие сообщения с использованием оператора switch, а функция renderGameBoard динамически создаёт HTML-элементы для каждой карточки на основе полученного состояния. Для предотвращения некорректных действий во время анимации переворота карт используются флаги isMyTurn и waitingForFlipReset. Всплывающие уведомления о ходе игры (например, «Ваш ход!», «Игрок нашёл пару») отображаются в правом нижнем углу экрана и автоматически исчезают через 2 секунды. На рисунках 3.6, 3.7 представлены всплывающие уведомления.

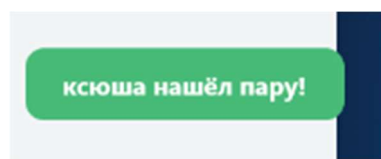


Рисунок 3.6 – Всплывающие уведомления

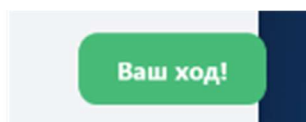


Рисунок 3.7 – Всплывающие уведомления

3.4 Дополнительные функции

В приложении реализован ряд дополнительных функций, расширяющих возможности пользователя.

При разрыве WebSocket-соединения клиент автоматически предпринимает попытки переподключения. Реализовано до пяти попыток с увеличивающейся задержкой: 2, 4, 6, 8 и 10 секунд. После успешного переподключения клиент отправляет команду force_update для получения актуального состояния игры.

Для идентификации клиента при переподключении используется уникальный идентификатор connectionId, который генерируется при первом

запуске приложения и сохраняется в localStorage браузера. При повторных запусках идентификатор загружается из хранилища, что позволяет серверу связать повторное подключение с уже существующим игроком в комнате.

Для потокобезопасной отправки сообщений клиентам на сервере используется механизм блокировок с помощью SemaphoreSlim. Это предотвращает коллизии при одновременной отправке нескольких сообщений одному клиенту из разных потоков.

4 ТЕСТИРОВАНИЕ

В процессе разработки веб-приложения было проведено тестирование, направленное на проверку корректности работы всех основных компонентов: серверной логики, сетевого взаимодействия и пользовательского интерфейса. Тестирование проводилось методом «белого ящика» с анализом исходного кода, а также функциональное тестирование в реальных условиях эксплуатации [10].

Тестирование сетевого взаимодействия. Для проверки корректности обмена данными между клиентом и сервером использовался анализатор трафика Wireshark. Захват производился на интерфейсе loopback с фильтром `tcp.port == 8081 or tcp.port == 5000`, что позволило изолировать трафик разрабатываемого приложения от других программ.

На рисунках 4.1, 4.2, 4.3 показан процесс установки WebSocket-соединения. Видно TCP-рукопожатие (пакеты SYN, SYN-ACK, ACK), после которого следует HTTP-запрос от клиента с заголовками `Upgrade: websocket` и `Connection: Upgrade`, содержащий ключ `Sec-WebSocket-Key`. Сервер отвечает кодом 101 Switching Protocols с вычисленным ключом `Sec-WebSocket-Accept`, после чего соединение переходит в режим постоянного двустороннего обмена.

495	2.803988	127.0.0.1	127.0.0.1	TCP	56	62598 → 8081 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
496	2.804116	127.0.0.1	127.0.0.1	TCP	56	8081 → 62598 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
497	2.804224	127.0.0.1	127.0.0.1	TCP	44	62598 → 8081 [ACK] Seq=1 Ack=1 Win=2619648 Len=0
498	2.806047	127.0.0.1	127.0.0.1	HTTP	562	GET / HTTP/1.1
499	2.806198	127.0.0.1	127.0.0.1	TCP	44	8081 → 62598 [ACK] Seq=1 Ack=519 Win=2619648 Len=0
500	2.831072	127.0.0.1	127.0.0.1	HTTP	173	HTTP/1.1 101 Switching Protocols
501	2.831130	127.0.0.1	127.0.0.1	TCP	44	62598 → 8081 [ACK] Seq=519 Ack=130 Win=2619648 Len=0

Рисунок 4.1 – Установка WebSocket-соединения

```
> GET / HTTP/1.1\r\n
Host: localhost:8081\r\n
Connection: Upgrade\r\n
Pragma: no-cache\r\n
Cache-Control: no-cache\r\n
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 \r\n
Upgrade: websocket\r\n
Origin: http://localhost:5000\r\n
Sec-WebSocket-Version: 13\r\n
Accept-Encoding: gzip, deflate, br, zstd\r\n
Accept-Language: ru-RU,ru;q=0.9,en-US;q=0.8,en;q=0.7\r\n
Sec-WebSocket-Key: DL6Ig2lUuMhIMjEZ1KR0GQ==\r\n
Sec-WebSocket-Extensions: permessage-deflate; client_max_window_bits\r\n
\r\n
```

Рисунок 4.2 – Установка WebSocket-соединения

```

HTTP/1.1 101 Switching Protocols\r\n
Upgrade: websocket\r\n
Connection: Upgrade\r\n
Sec-WebSocket-Accept: lYPxdUSkiXZqAEqxsVs178qXK44=\r\n
\r\n
[Request in frame: 498]
[Time since request: 25.025000 milliseconds]
[Request URI: /]
[Full request URI: http://localhost:8081/]

```

Рисунок 4.3 – Установка WebSocket-соединения

На рисунке 4.4 представлено содержимое WebSocket-сообщения с JSON-данными от клиента. В данном примере клиент отправляет команду `create_room` для создания новой игровой комнаты. Пакет идентифицируется как `WebSocket Text [FIN] [MASKED]`, что указывает на замаскированные данные от клиента к серверу. В теле сообщения виден JSON с полями `type`, `connectionId`, `playerName` и `boardSize`.

```

2048 123.269131 127.0.0.1 127.0.0.1 WebSo... 145 WebSocket Text [FIN] [MASKED]
> Transmission Control Protocol, Src Port: 62631, Dst Port: 8081 Seq: 670, Ack: 204, Len: 101
  v WebSocket
    1... .... = Fin: True
    .000 .... = Reserved: 0x0
    .... 0001 = Opcode: Text (1)
    1... .... = Mask: True
    .101 1111 = Payload length: 95
    Masking-Key: 108d8ae8
    Masked payload
  > Payload
  v Line-based text data (1 lines)
    {"type":"create_room","connectionId":"player_k0cv9i63o5","playerName":"лепа","boardSize":4}

```

Рисунок 4.4 – Отправка клиентом команды `create_room`

На рисунке 4.5 показан ответ сервера на команду создания комнаты. Сервер отправляет сообщение `room_created` с кодом комнаты и размером поля. Данный пакет не имеет маскировки (`[FIN]` без `MASKED`), что соответствует спецификации `WebSocket`, согласно которой сервер не маскирует отправляемые данные.

2050	123.275207	127.0.0.1	127.0.0.1	WebSo...	101	WebSocket Text [FIN]
> Transmission Control Protocol, Src Port: 8081, Dst Port: 62631, Seq: 204, Ack: 771, Len: 57						
v WebSocket						
1... = Fin: True						
.000 = Reserved: 0x0						
.... 0001 = Opcode: Text (1)						
0... = Mask: False						
.011 0111 = Payload length: 55						
> Payload						
v Line-based text data (1 lines)						
{ "type": "room_created", "roomCode": "NDG6", "boardSize": 4 }						

Рисунок 4.5 – Ответ сервера с подтверждением создания комнаты

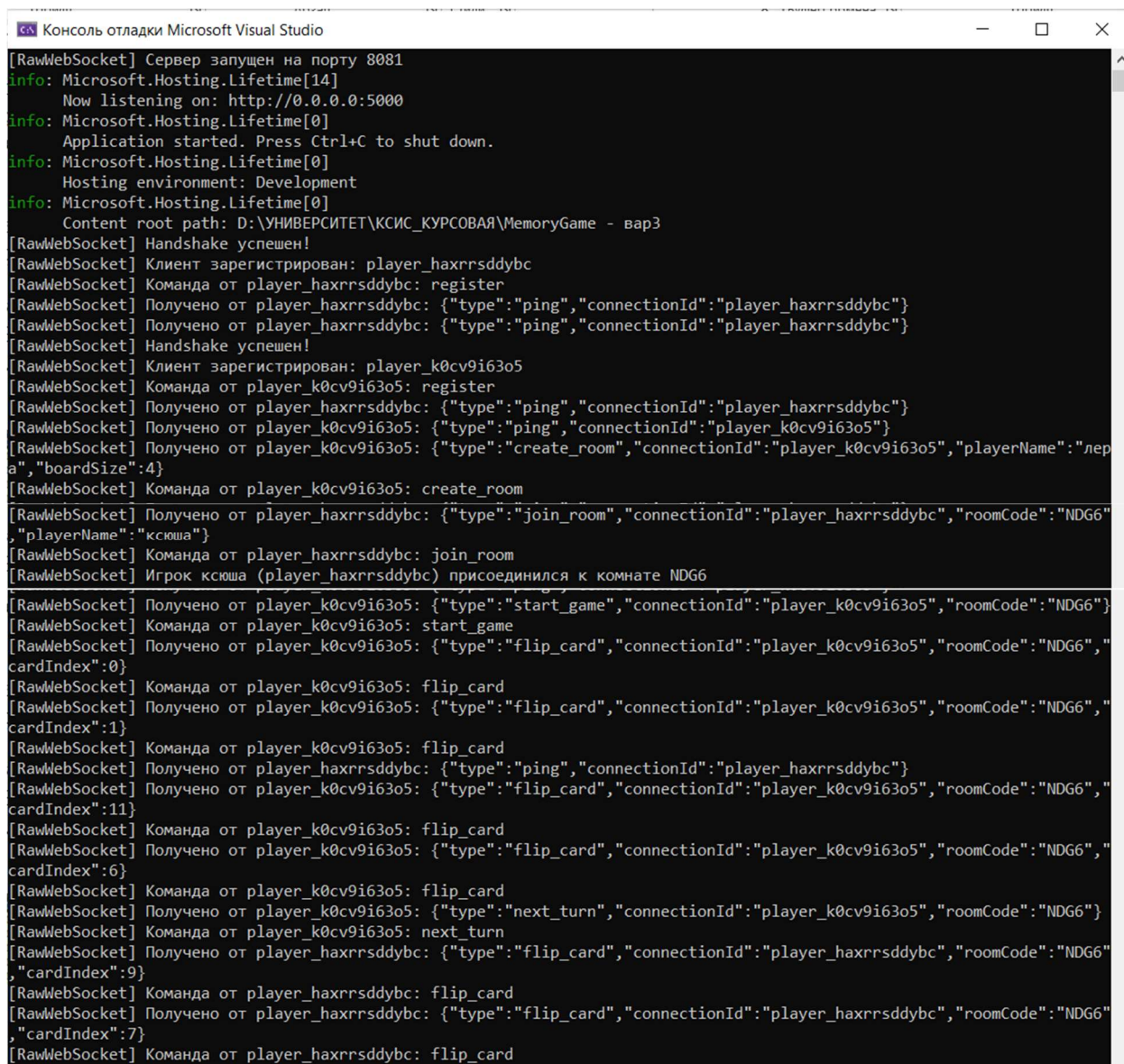
При закрытии вкладки браузера происходит корректное завершение TCP-соединения. На рисунке 4.6 виден обмен FIN-пакетами, после чего соединение закрывается.

3916	429.713696	127.0.0.1	127.0.0.1	TCP	44	8081 → 62631 [FIN, ACK] Seq=14304 Ack=3180 Win=2617088 Len=0
3917	429.713720	127.0.0.1	127.0.0.1	TCP	44	62631 → 8081 [ACK] Seq=3180 Ack=14305 Win=2605312 Len=0
3918	429.713856	127.0.0.1	127.0.0.1	TCP	44	62631 → 8081 [FIN, ACK] Seq=3180 Ack=14305 Win=2605312 Len=0
3919	429.713939	127.0.0.1	127.0.0.1	TCP	44	8081 → 62631 [ACK] Seq=14305 Ack=3181 Win=2617088 Len=0

Рисунок 4.6 – Корректное закрытие соединения

В ходе анализа было установлено, что все сообщения между клиентом и сервером передаются в соответствии со спецификацией RFC 6455. Клиентские сообщения правильно маскируются, серверные — отправляются без маскировки. Формат JSON соблюдается, все поля передаются корректно.

Диагностика работы сервера. Для отладки и контроля работы серверной части в консоль выводятся диагностические сообщения. На рисунке 4.7 показан пример вывода логов при запуске сервера и подключении игроков.

The image shows a screenshot of the 'Консоль отладки Microsoft Visual Studio' (Visual Studio Debug Console). The console displays a series of log messages from a WebSocket server. The messages are color-coded: green for 'info' and white for other log types. The logs show the server starting on port 8081, listening on http://0.0.0.0:5000, and the application starting. Two clients, 'player_haxrrsddybc' and 'player_k0cv9i63o5', register and send 'ping' messages. Then, 'player_k0cv9i63o5' sends a 'create_room' message with parameters like 'playerName', 'boardSize', and 'roomCode'. 'player_haxrrsddybc' then sends a 'join_room' message. The logs continue with 'start_game', 'flip_card', and 'next_turn' messages, showing the progression of a game session. The console window has standard Windows window controls at the top right.

```
[RawWebSocket] Сервер запущен на порту 8081
info: Microsoft.Hosting.Lifetime[14]
  Now listening on: http://0.0.0.0:5000
info: Microsoft.Hosting.Lifetime[0]
  Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
  Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
  Content root path: D:\УНИВЕРСИТЕТ\КСИС_КУРСОВАЯ\MemoryGame - var3
[RawWebSocket] Handshake успешен!
[RawWebSocket] Клиент зарегистрирован: player_haxrrsddybc
[RawWebSocket] Команда от player_haxrrsddybc: register
[RawWebSocket] Получено от player_haxrrsddybc: {"type":"ping","connectionId":"player_haxrrsddybc"}
[RawWebSocket] Получено от player_haxrrsddybc: {"type":"ping","connectionId":"player_haxrrsddybc"}
[RawWebSocket] Handshake успешен!
[RawWebSocket] Клиент зарегистрирован: player_k0cv9i63o5
[RawWebSocket] Команда от player_k0cv9i63o5: register
[RawWebSocket] Получено от player_haxrrsddybc: {"type":"ping","connectionId":"player_haxrrsddybc"}
[RawWebSocket] Получено от player_k0cv9i63o5: {"type":"ping","connectionId":"player_k0cv9i63o5"}
[RawWebSocket] Получено от player_k0cv9i63o5: {"type":"create_room","connectionId":"player_k0cv9i63o5","playerName":"леп
a","boardSize":4}
[RawWebSocket] Команда от player_k0cv9i63o5: create_room
[RawWebSocket] Получено от player_haxrrsddybc: {"type":"join_room","connectionId":"player_haxrrsddybc","roomCode":"NDG6",
,"playerName":"ксюша"}
[RawWebSocket] Команда от player_haxrrsddybc: join_room
[RawWebSocket] Игрок ксюша (player_haxrrsddybc) присоединился к комнате NDG6
[RawWebSocket] Получено от player_k0cv9i63o5: {"type":"start_game","connectionId":"player_k0cv9i63o5","roomCode":"NDG6"}
[RawWebSocket] Команда от player_k0cv9i63o5: start_game
[RawWebSocket] Получено от player_k0cv9i63o5: {"type":"flip_card","connectionId":"player_k0cv9i63o5","roomCode":"NDG6",
"cardIndex":0}
[RawWebSocket] Команда от player_k0cv9i63o5: flip_card
[RawWebSocket] Получено от player_k0cv9i63o5: {"type":"flip_card","connectionId":"player_k0cv9i63o5","roomCode":"NDG6",
"cardIndex":1}
[RawWebSocket] Команда от player_k0cv9i63o5: flip_card
[RawWebSocket] Получено от player_haxrrsddybc: {"type":"ping","connectionId":"player_haxrrsddybc"}
[RawWebSocket] Получено от player_k0cv9i63o5: {"type":"flip_card","connectionId":"player_k0cv9i63o5","roomCode":"NDG6",
"cardIndex":11}
[RawWebSocket] Команда от player_k0cv9i63o5: flip_card
[RawWebSocket] Получено от player_k0cv9i63o5: {"type":"flip_card","connectionId":"player_k0cv9i63o5","roomCode":"NDG6",
"cardIndex":6}
[RawWebSocket] Команда от player_k0cv9i63o5: flip_card
[RawWebSocket] Получено от player_k0cv9i63o5: {"type":"next_turn","connectionId":"player_k0cv9i63o5","roomCode":"NDG6"}
[RawWebSocket] Команда от player_k0cv9i63o5: next_turn
[RawWebSocket] Получено от player_haxrrsddybc: {"type":"flip_card","connectionId":"player_haxrrsddybc","roomCode":"NDG6",
,"cardIndex":9}
[RawWebSocket] Команда от player_haxrrsddybc: flip_card
[RawWebSocket] Получено от player_haxrrsddybc: {"type":"flip_card","connectionId":"player_haxrrsddybc","roomCode":"NDG6",
,"cardIndex":7}
[RawWebSocket] Команда от player_haxrrsddybc: flip_card
```

Рисунок 4.7 – Диагностические сообщения сервера в консоли

Логирование включает следующие события: запуск WebSocket-сервера на порту 8081, регистрация нового клиента с его идентификатором, создание новой комнаты с указанием кода и размера поля, подключение игрока к комнате, начало игры, открытие карточек, найденные пары, смена хода, завершение игры, отключение клиента. Это позволяет в реальном времени отслеживать состояние сервера и при необходимости выявлять ошибки.

Тестирование игровой логики. Проверка игровой механики проводилась вручную путём создания игровой сессии с двумя и более игроками. Тестирование включало следующие сценарии: создание комнаты с различными размерами поля (4×4, 6×6, 8×8), подключение игроков по коду комнаты, запуск игры создателем комнаты, открытие карточек игроками по очереди, проверка правильности определения совпадающих пар, начисление очков при нахождении пары, переворот карточек при несовпадении, переход хода к следующему игроку после несовпадения, определение победителя по

окончании игры, перезапуск игры создателем комнаты, обработка отключения игрока во время игры.

В процессе тестирования были выявлены и устранены следующие недостатки. При одновременном подключении нескольких игроков наблюдалась задержка обновления списка игроков на экране ожидания. Проблема была решена добавлением немедленной рассылки сообщения `players_updated` после каждого подключения. При открытии второй карточки во время анимации переворота первой возникала возможность повторного клика, что приводило к некорректному состоянию. Проблема была устранена введением флага `waitingForFlipReset`, блокирующего клики до завершения анимации. При отключении игрока во время игры сервер не всегда корректно удалял его из комнаты, что приводило к «зависанию» хода. Проблема была решена добавлением метода `HandleDisconnect`, который при разрыве соединения удаляет игрока и при необходимости переключает ход.

Тестирование пользовательского интерфейса. Проверка интерфейса включала оценку корректности отображения на различных устройствах и в различных браузерах. Тестирование проводилось в браузерах Google Chrome, Mozilla Firefox на операционных системах Windows и на мобильных устройствах под управлением iOS. На рисунках 4.8, 4.9 представлен экран лобби на мобильном устройстве.

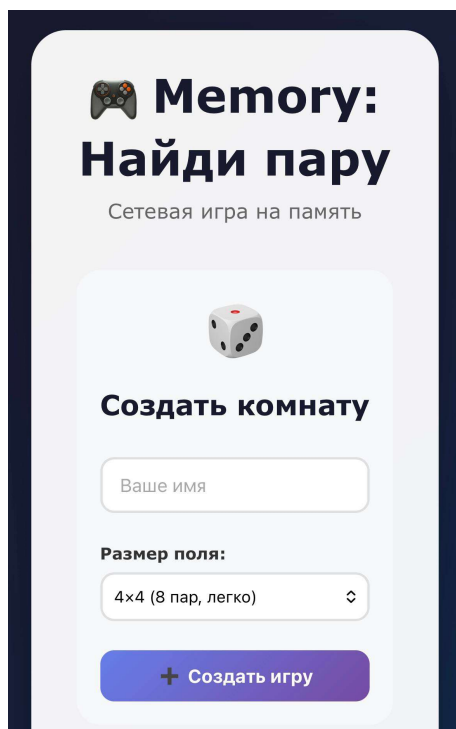


Рисунок 4.8 – Экран лобби на мобильном устройстве

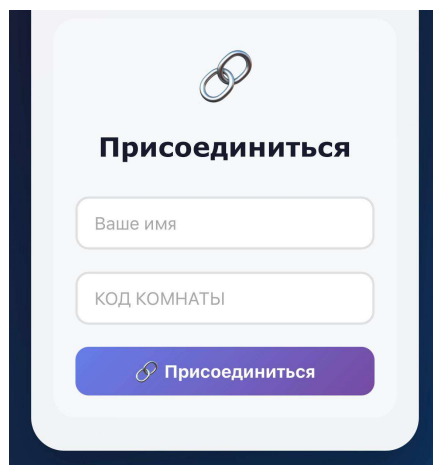


Рисунок 4.9 – Экран лобби на мобильном устройстве

На рисунке 4.10 показан экран ожидания на телефоне. Код комнаты и список игроков отображаются корректно, кнопки имеют достаточный размер для нажатия пальцем.

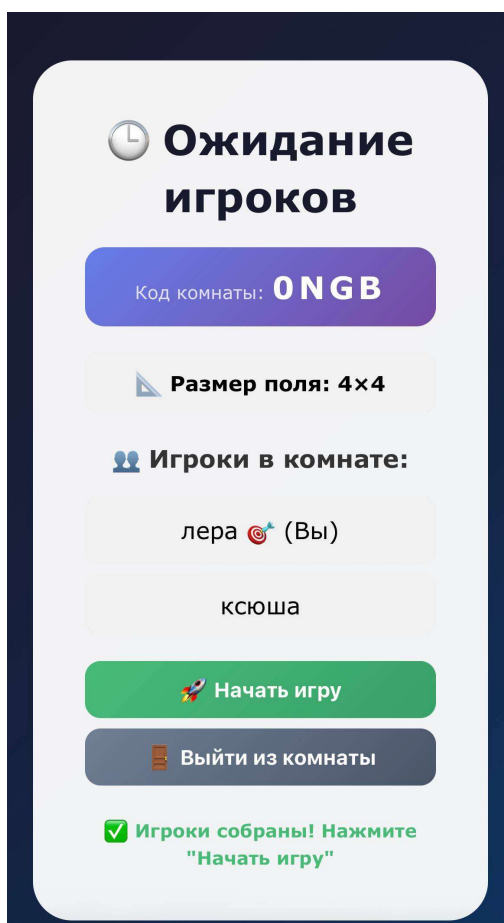


Рисунок 4.10 – Экран ожидания на мобильном устройстве

На рисунке 4.11 изображён игровой экран на телефоне. Карточки автоматически уменьшаются, чтобы поместиться на экране, при этом остаются различимыми и удобными для нажатия.

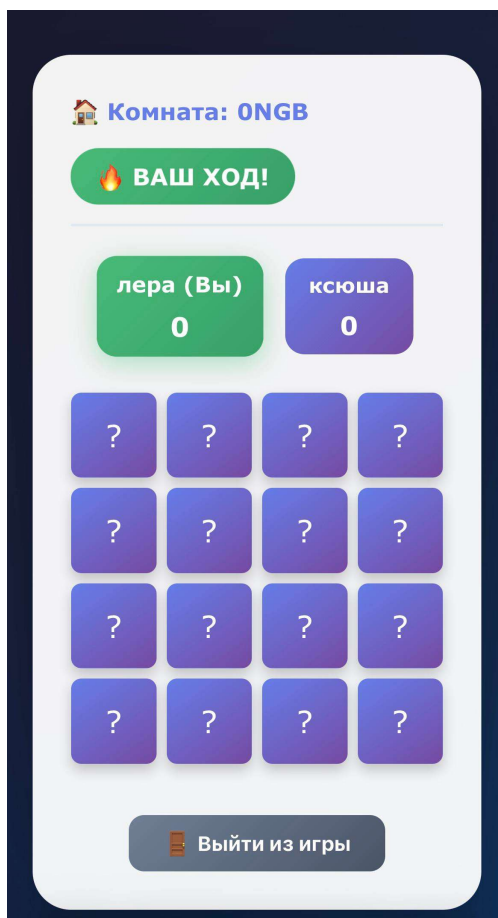


Рисунок 4.11 – Игровой экран на мобильном устройстве

В ходе тестирования установлено, что CSS Grid корректно перестраивает игровое поле в зависимости от размера экрана. Медиа-запросы срабатывают при ширине экрана менее 768 пикселей: формы на экране лобби выстраиваются вертикально, отступы между карточками уменьшаются, размер шрифта на кнопках остаётся читаемым. Все элементы интерфейса имеют достаточный размер для комфортного взаимодействия с помощью сенсорного управления.

Нагрузочное тестирование. Для проверки стабильности работы приложения при максимальной нагрузке было проведено тестирование с четырьмя одновременно подключёнными игроками (максимальное количество, поддерживаемое комнатой). Тестирование показало, что сервер стабильно обрабатывает все входящие сообщения, среднее время отклика составляет менее 100 миллисекунд. При одновременной отправке нескольких команд от разных игроков коллизий не возникает благодаря использованию потокобезопасных коллекций и механизма блокировок.

Тестирование обработки ошибок. Были протестированы следующие ошибочные ситуации: попытка подключения к несуществующей комнате (сервер возвращает сообщение об ошибке), попытка начать игру с одним игроком (кнопка «Начать игру» неактивна), попытка открыть уже найденную карточку (запрос игнорируется), попытка сделать ход не в свою очередь (клиент блокирует действие и выводит уведомление), разрыв соединения во время игры (сервер корректно удаляет игрока, игра продолжается для остальных). Все ошибочные ситуации обрабатываются корректно, приложение не завершает работу аварийно и сохраняет стабильное состояние.

Таким образом, проведённое тестирование подтвердило, что разработанное веб-приложение функционирует корректно, соответствует заявленным требованиям и готово к эксплуатации.

5 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

5.1 Описание пользовательского интерфейса

Пользовательский интерфейс приложения состоит из трёх основных экранов: экран лобби, экран ожидания и игровой экран. Все экраны выполнены в тёмно-синей цветовой гамме с градиентным фоном, что обеспечивает комфортное восприятие при длительной игре.

Экран лобби (рисунок 5.1) является главным экраном приложения. Он содержит две формы для взаимодействия с игроком. Левая форма «Создать комнату» предназначена для создания новой игровой сессии. В ней расположены поле для ввода имени игрока, выпадающий список для выбора размера поля и кнопка «Создать игру». Правая форма «Присоединиться» предназначена для подключения к уже созданной комнате. В ней расположены поле для ввода имени игрока, поле для ввода кода комнаты и кнопка «Присоединиться». Поля ввода имеют подсветку при наведении курсора. При попытке создать или присоединиться к игре без заполнения имени пользователя выводится предупреждение (рисунок 5.2).

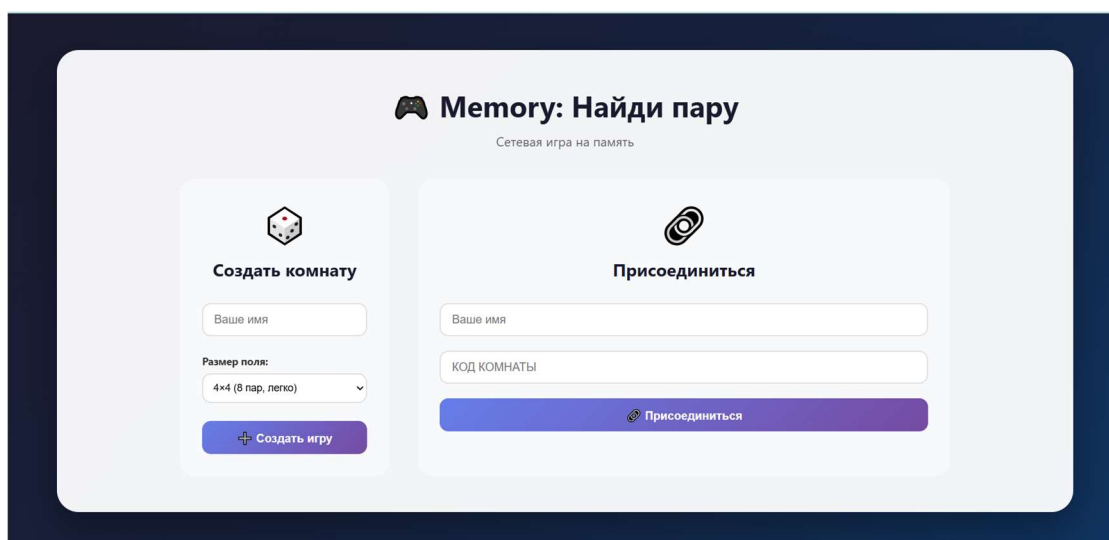


Рисунок 5.1 – Экран лобби

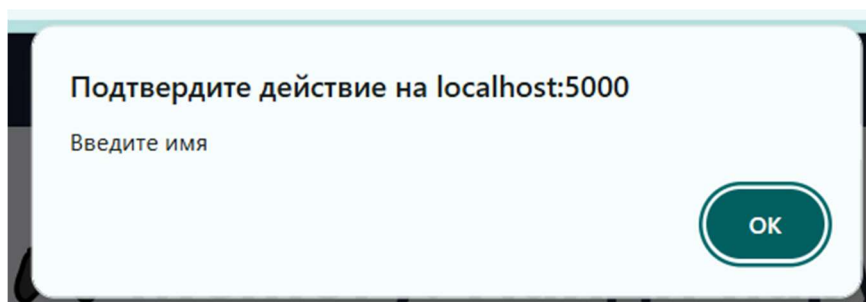


Рисунок 5.2 – Предупреждение о незаполненном имени

Экран ожидания (рисунок 5.3) отображается после создания комнаты или успешного подключения к существующей. В верхней части экрана крупным шрифтом на цветном фоне показан уникальный 4-значный код комнаты, который необходимо передать другим игрокам для подключения. Ниже отображается выбранный размер поля. В центральной части расположен список подключившихся игроков, где текущий игрок помечен подписью «(Вы)». Для создателя комнаты отображается кнопка «Начать игру». Внизу экрана расположена кнопка «Выйти из комнаты» для возврата в лобби. При подключении новых игроков список обновляется автоматически, а информационный текст меняется на «Игроки собраны! Нажмите "Начать игру"».

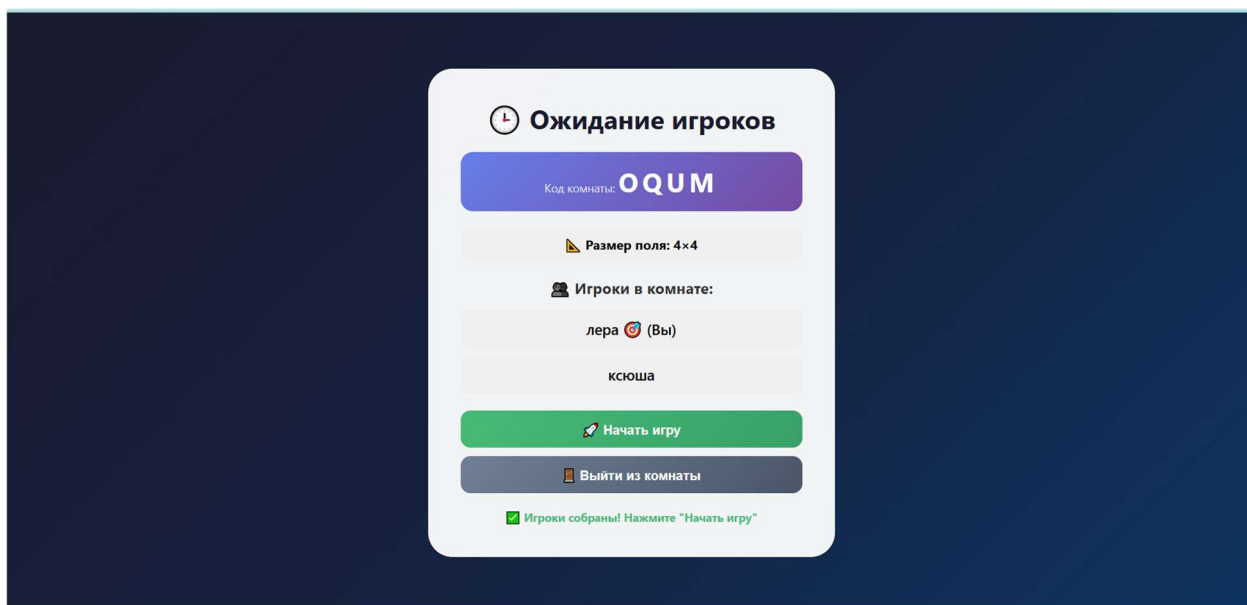


Рисунок 5.3 – Экран ожидания

Игровой экран (рисунок 5.4) отображается после нажатия кнопки «Начать игру». В верхней панели показан код комнаты и индикатор текущего хода. Когда наступает ход игрока, индикатор подсвечивается зелёным цветом и пульсирует. В левой части экрана расположена панель игроков с именами всех участников и их текущим счётом. Текущий игрок визуально выделяется зелёным фоном. Центральную часть экрана занимает игровое поле в виде сетки карточек. Размер сетки зависит от выбранного перед игрой размера поля (4×4, 6×6 или 8×8). Внизу экрана расположена кнопка «Выйти из игры».

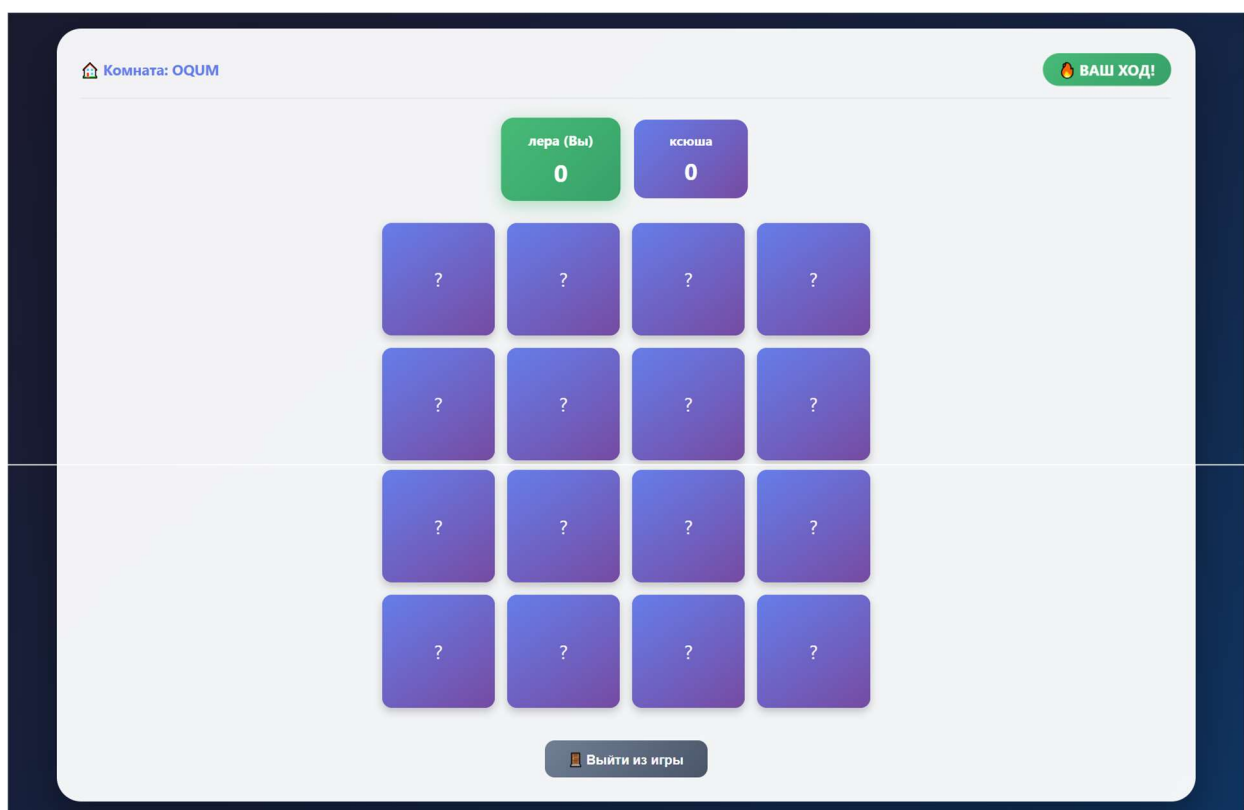


Рисунок 5.4 – Игровой экран

Карточки в закрытом состоянии выглядят как фиолетово-синие квадраты с вопросительным знаком в центре. При нажатии на карточку она переворачивается, и на ней появляется изображение. Когда игрок находит пару, обе карточки отмечаются зелёным фоном и символом «✓», после чего становятся недоступными для дальнейших нажатий.

Модальное окно окончания игры (рисунок 5.5) появляется после завершения партии. Окно содержит иконку кубка в верхней части, сообщение о победителе (или ничье), а также две кнопки: «Сыграть ещё раз» (доступна только создателю комнаты) и «Заккрыть». При нажатии на «Сыграть ещё раз» начинается новая игра без повторного подключения игроков.

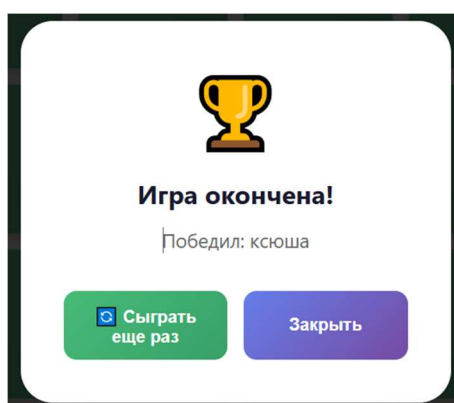


Рисунок 5.5 – Модальное окно окончания игры

Интерфейс адаптирован для использования на мобильных устройствах. При уменьшении ширины экрана формы на экране лобби выстраиваются вертикально, а размер карточек на игровом поле автоматически уменьшается для удобства нажатия пальцем.

5.2 Управление игровым процессом

Раздел описывает шаги по созданию и подключению к игре, а также основные действия во время игрового процесса.

Для создания новой игры пользователь должен выполнить следующие шаги:

- 1 На экране лобби ввести своё имя в поле «Ваше имя» формы «Создать комнату».
- 2 Выбрать размер игрового поля из выпадающего списка (4×4, 6×6 или 8×8).
- 3 Нажать кнопку «Создать игру».
- 4 После создания комнаты система отобразит экран ожидания с уникальным 4-значным кодом комнаты.
- 5 Передать полученный код другим игрокам для подключения.

Для подключения к уже созданной комнате пользователь должен выполнить следующие шаги:

- 1 На экране лобби ввести своё имя в поле «Ваше имя» формы «Присоединиться».
- 2 Ввести 4-значный код комнаты в поле «Код комнаты».
- 3 Нажать кнопку «Присоединиться».
- 4 После успешного подключения система отобразит экран ожидания, где будет виден список подключившихся игроков.

Игра может быть начата только создателем комнаты при наличии не менее двух игроков в комнате. Создатель комнаты должен нажать кнопку «Начать игру» на экране ожидания. После этого всем игрокам отобразится игровой экран, и начнётся первый ход.

Игроки ходят по очереди. Текущий игрок определяется индикатором хода (зелёная пульсирующая подсветка) и выделением на панели игроков. В свой ход игрок может открыть две карточки:

– Для открытия карточки необходимо нажать на закрытую карточку на игровом поле. При нажатии карточка переворачивается лицевой стороной вверх, на ней появляется изображение.

– Если игрок открыл две карточки, система проверяет совпадение изображений. Если изображения совпадают, карточки остаются открытыми, игрок получает одно очко, и его ход продолжается. Если изображения не совпадают, карточки автоматически переворачиваются обратно через 1,2 секунды, и ход переходит к следующему игроку.

Во время анимации переворота карточек нажатия на другие карточки игнорируются, чтобы предотвратить некорректные действия.

Игра завершается, когда все пары карточек найдены. После этого отображается модальное окно с результатами, в котором указан победитель или объявлена ничья. Создатель комнаты может нажать кнопку «Сыграть ещё раз» для начала новой игры без повторного подключения игроков. Остальные игроки могут нажать кнопку «Закрыть» для возврата в лобби.

В любой момент игрок может покинуть игру, нажав кнопку «Выйти из комнаты» на экране ожидания или кнопку «Выйти из игры» на игровом экране. При выходе игрок удаляется из комнаты, а остальные игроки получают соответствующее уведомление.

5.3 Правила игры

Игра «Memory: Найди пару» представляет собой карточную игру на развитие памяти, в которой игроки по очереди открывают карточки, пытаясь найти совпадающие пары.

Цель игры: набрать наибольшее количество очков, находя пары одинаковых изображений на игровом поле.

Подготовка к игре: перед началом игры игроки выбирают размер поля. Доступны три варианта: 4×4 (8 пар, рекомендуется для новичков), 6×6 (18 пар, средний уровень сложности), 8×8 (32 пары, для опытных игроков). Все карточки на поле располагаются рубашкой вверх в случайном порядке.

Ход игры: игроки ходят по очереди. В свой ход игрок открывает две карточки. Если изображения на карточках совпадают, игрок забирает пару себе и получает одно очко. При этом игрок получает право сделать ещё один ход. Если изображения не совпадают, карточки переворачиваются обратно, и ход переходит к следующему игроку.

Подсчёт очков: каждая найденная пара приносит игроку одно очко. Очки отображаются на панели игроков рядом с именем каждого участника.

Окончание игры: игра заканчивается, когда все пары карточек найдены. Победителем объявляется игрок, набравший наибольшее количество очков. Если несколько игроков набрали одинаковое количество очков, объявляется ничья.

Стратегические советы: внимательно следите за карточками, которые открывают соперники, – это помогает запомнить расположение изображений. Старайтесь запоминать не только изображения, но и их позиции на поле. Если вы нашли пару и получаете дополнительный ход, используйте его для поиска следующей пары, пока вы помните расположение других карточек.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта разработано веб-приложение, реализующее сетевую версию игры «Memory: Найди пару». Приложение построено по клиент-серверной архитектуре и позволяет игрокам создавать игровые комнаты, присоединяться к ним по уникальному коду и соревноваться в поиске пар карточек в реальном времени через браузер без установки дополнительного программного обеспечения.

Серверная часть реализована на языке C# с использованием платформы ASP.NET Core. Для обеспечения двусторонней связи в реальном времени разработан собственный WebSocket-сервер, работающий по стандарту RFC 6455. Сервер обрабатывает подключения игроков, управляет игровыми комнатами и выполняет всю игровую логику: генерацию игрового поля заданного размера (4×4, 6×6 или 8×8), проверку совпадения карточек, подсчёт очков, смену хода и определение победителя. Все сообщения между клиентом и сервером передаются в формате JSON, что обеспечивает удобство отладки и гибкость при расширении функционала.

Клиентская часть разработана на HTML, CSS и JavaScript. Интерфейс представляет собой одностраничное приложение с динамической сменой трёх экранов: экран лобби (создание или подключение к комнате), экран ожидания (отображение кода комнаты и списка игроков) и игровой экран (поле с карточками, панель игроков с счетами, индикатор текущего хода). Реализована адаптивная вёрстка, обеспечивающая корректное отображение на устройствах с разной шириной экрана, а также анимации появления экранов и переворота карточек. Для предотвращения некорректных действий во время анимации и при ходе соперника реализована система блокировки интерфейса.

В приложении реализован ряд дополнительных функций. Клиент автоматически предпринимает несколько попыток переподключения при разрыве соединения и после успешного переподключения запрашивает актуальное состояние игры. Для идентификации клиента используется уникальный идентификатор, сохраняемый в локальном хранилище браузера. На сервере реализована потокобезопасная отправка сообщений клиентам, предотвращающая коллизии при одновременной отправке нескольких сообщений.

Проведённое тестирование подтвердило корректность работы всех компонентов приложения. С помощью анализатора трафика Wireshark проверены установка WebSocket-соединения, обмен данными в формате JSON и корректное закрытие соединения. Выявленные в процессе тестирования проблемы (задержка обновления списка игроков, повторные клики во время анимации переворота карт, некорректное удаление игрока при отключении) были устранены. Нагрузочное тестирование с максимальным количеством одновременно подключённых игроков показало стабильную работу сервера с минимальным временем отклика.

Разработанное приложение полностью соответствует поставленным требованиям. Все функциональные задачи решены: обеспечена организация игровых сессий, генерация игрового поля, поддержка многопользовательского режима с поочерёдным выполнением ходов, синхронизация состояния в реальном времени, подсчёт результатов и удобный адаптивный интерфейс. Приложение готово к практическому использованию для игры в локальной сети.

В процессе выполнения курсового проекта от использования ASP.NET Core SignalR, указанного в задании, пришлось отказаться в пользу ручной реализации WebSocket-сервера. Это связано с тем, что SignalR скрывает детали низкоуровневой работы протокола, тогда как самостоятельная реализация handshake, фрейминга и маскировки данных позволила глубже разобраться в принципах сетевого взаимодействия, получить полный контроль над форматом сообщений и продемонстрировать навыки разработки на уровне TCP-сокетов.

Перспективы дальнейшего развития проекта включают добавление системы регистрации и авторизации с сохранением статистики игроков, реализацию чата в игровой комнате, добавление возможности выбора тематики карточек, а также создание рейтинговой системы для отслеживания лучших результатов.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

- [1] Доманов, А. Т. Стандарт предприятия СТП 01-2024 / А. Т. Доманов, Н. И. Сорока. – Минск : БГУИР, 2024.
- [2] Игра Память (Memory) онлайн [Электронный ресурс]. – Режим доступа : <https://pairs.one/ru>.
- [3] Тренажер Найди Пару (Memory) [Электронный ресурс]. – Режим доступа : <https://mozgotren.com/trening/trenagor/memory>.
- [4] Microsoft Learn. Обзор ASP.NET Core [Электронный ресурс]. – Режим доступа : <https://learn.microsoft.com/ru-ru/aspnet/core/introduction-to-aspnet-core>.
- [5] MDN Web Docs. Writing a WebSocket server in C# [Электронный ресурс]. – Режим доступа : https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API/Writing_WebSocket_server.
- [6] MDN Web Docs. Веб-технологии для разработчиков [Электронный ресурс]. – Режим доступа : <https://developer.mozilla.org/ru/docs/Web>.
- [7] Fette, I. The WebSocket Protocol, RFC 6455 / I. Fette, A. Melnikov. – 2011. – Режим доступа : <https://datatracker.ietf.org/doc/html/rfc6455>.
- [8] Кнут, Д. Э. Искусство программирования. В 7 т. Т. 2. Получисленные алгоритмы / Д. Э. Кнут. – М. : Вильямс, 2018. – 832 с.
- [9] Флэнаган, Д. JavaScript. Полное руководство / Д. Флэнаган. – М. : Диалектика, 2022. – 720 с.
- [10] Введение в тестирование [Электронный ресурс]. – Режим доступа : <https://habr.com/ru/companies/nauka/articles/874472>.

ПРИЛОЖЕНИЕ А
(обязательное)
Листинг кода с комментариями

```
using System.Net.Sockets;
using System.Net;
using System.Security.Cryptography;
using System.Text;
using System.Text.Json;
using System.Collections.Concurrent;
using MemoryGame.Services;
using MemoryGame.Models;
namespace MemoryGame.Handlers
{
    public class RawWebSocketHandler
    {
        private readonly GameRoomManager _roomManager;
        private static readonly ConcurrentDictionary<string, TcpClient> _clients = new();
        private static readonly ConcurrentDictionary<string, string> _connectionToRoom = new();
        private static readonly ConcurrentDictionary<string, SemaphoreSlim> _clientLocks =
new();
        public RawWebSocketHandler(GameRoomManager roomManager)
        {
            _roomManager = roomManager;
        }
        // Запускает TCP сервер на указанном порту и принимает входящие соединения
        public async Task StartServer(int port)
        {
            var listener = new TcpListener(IPAddress.Any, port);
            listener.Start();
            Console.WriteLine($"[RawWebSocket] Сервер запущен на порту {port}");
            while (true)
            {
                var client = await listener.AcceptTcpClientAsync();
                _ = Task.Run(() => HandleClient(client));
            }
        }
        // Обработывает одного клиента: WebSocket handshake, регистрация, цикл сообщений
        private async Task HandleClient(TcpClient tcpClient)
        {
            var stream = tcpClient.GetStream();
            string connectionId = null;
            try
            {
                // WebSocket Handshake
                byte[] buffer = new byte[4096];
                int bytesRead = await stream.ReadAsync(buffer, 0, buffer.Length);
                string httpRequest = Encoding.UTF8.GetString(buffer, 0, bytesRead);
                string wsKey = ExtractWebSocketKey(httpRequest);
                if (wsKey == null)
                {

```

```

        await SendHttpResponse(stream, 400, "Bad Request");
        return;
    }
    string acceptKey = ComputeWebSocketAccept(wsKey);
    string handshakeResponse =
        "HTTP/1.1 101 Switching Protocols\r\n" +
        "Upgrade: websocket\r\n" +
        "Connection: Upgrade\r\n" +
        $"Sec-WebSocket-Accept: {acceptKey}\r\n\r\n";
    await stream.WriteAsync(Encoding.ASCII.GetBytes(handshakeResponse));
    Console.WriteLine("[RawWebSocket] Handshake успешен!");
    // Регистрация клиента
    byte[] frameData = await ReadWebSocketFrame(stream);
    if (frameData == null) return;
    string firstMessage = Encoding.UTF8.GetString(frameData);
    var options = new JsonSerializerOptions { PropertyNameCaseInsensitive = true };
    var tempCommand = JsonSerializer.Deserialize<WebSocketCommand>(firstMessage,
options);
    if (tempCommand?.ConnectionId != null)
    {
        connectionId = tempCommand.ConnectionId;
        // При переподключении закрываем старое соединение
        if (_clients.TryRemove(connectionId, out var oldClient))
        {
            Console.WriteLine($"[RawWebSocket] Переподключение клиента:
{connectionId}, закрываем старое соединение");
            try { oldClient.Close(); } catch { }
        }
        _clients[connectionId] = tcpClient;
        _clientLocks[connectionId] = new SemaphoreSlim(1, 1);
        Console.WriteLine($"[RawWebSocket] Клиент зарегистрирован:
{connectionId}");
        await SendWebSocketFrame(stream, JsonSerializer.Serialize(new
        {
            type = "connected",
            connectionId = connectionId
        }));

        // Обрабатываем первое сообщение если это не ping
        if (tempCommand.Type != null && tempCommand.Type != "ping")
        {
            await ProcessMessage(connectionId, firstMessage);
        }
    }
    else
    {
        Console.WriteLine($"[RawWebSocket] Клиент не прислал ConnectionId,
отключаем");
        return;
    }
    // Основной цикл обработки сообщений
    while (tcpClient.Connected)

```

```

        {
            byte[] data = await ReadWebSocketFrame(stream);
            if (data == null) break;
            string message = Encoding.UTF8.GetString(data);
            Console.WriteLine($"[RawWebSocket] Получено от {connectionId}: {message}");
            await ProcessMessage(connectionId, message);
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[RawWebSocket] Ошибка клиента {connectionId ??
"unknown"}: {ex.Message}");
    }
    finally
    {
        {
            if (connectionId != null)
            {
                await HandleDisconnect(connectionId);
                _clients.TryRemove(connectionId, out _);
                if (_clientLocks.TryRemove(connectionId, out var sem))
                    sem.Dispose();
            }
            try { tcpClient.Close(); } catch { }
        }
    }
}
// Читает WebSocket фрейм согласно RFC 6455
private async Task<byte[]> ReadWebSocketFrame(NetworkStream stream)
{
    byte[] header = new byte[2];
    int read = await ReadExact(stream, header, 2);
    if (read < 2) return null;
    int opcode = header[0] & 0x0F;
    bool masked = (header[1] & 0x80) != 0;
    int payloadLen = header[1] & 0x7F;
    if (payloadLen == 126)
    {
        byte[] ext = new byte[2];
        await ReadExact(stream, ext, 2);
        payloadLen = (ext[0] << 8) | ext[1];
    }
    else if (payloadLen == 127)
    {
        byte[] ext = new byte[8];
        await ReadExact(stream, ext, 8);
        payloadLen = (int)((long)ext[0] << 56 | ((long)ext[1] << 48) |
            ((long)ext[2] << 40) | ((long)ext[3] << 32) |
            ((long)ext[4] << 24) | ((long)ext[5] << 16) |
            ((long)ext[6] << 8) | ext[7]);
    }
    byte[] mask = new byte[4];
    if (masked)
        await ReadExact(stream, mask, 4);
}

```

```

byte[] payload = new byte[payloadLen];
await ReadExact(stream, payload, payloadLen);
// Размаскировка XOR
if (masked)
{
    for (int i = 0; i < payloadLen; i++)
        payload[i] = (byte)(payload[i] ^ mask[i % 4]);
}
if (opcode == 8) return null; // Close frame
if (opcode == 9) return new byte[0]; // Ping frame
return payload;
}
// Отправляет WebSocket фрейм клиенту
private async Task SendWebSocketFrame(NetworkStream stream, string message)
{
    byte[] data = Encoding.UTF8.GetBytes(message);
    using var ms = new MemoryStream();
    ms.WriteByte(0x81); // FIN=1, Opcode=1 (text)
    if (data.Length <= 125)
    {
        ms.WriteByte((byte)data.Length);
    }
    else if (data.Length <= 65535)
    {
        ms.WriteByte(126);
        ms.WriteByte((byte)(data.Length >> 8));
        ms.WriteByte((byte)(data.Length & 0xFF));
    }
    else
    {
        ms.WriteByte(127);
        long len = data.Length;
        for (int i = 7; i >= 0; i--)
            ms.WriteByte((byte)((len >> (i * 8)) & 0xFF));
    }
    ms.Write(data, 0, data.Length);
    byte[] frame = ms.ToArray();
    await stream.WriteAsync(frame, 0, frame.Length);
}
// Читает точное количество байт из потока
private async Task<int> ReadExact(NetworkStream stream, byte[] buffer, int count)
{
    int offset = 0;
    while (offset < count)
    {
        int read = await stream.ReadAsync(buffer, offset, count - offset);
        if (read == 0) return offset;
        offset += read;
    }
    return offset;
}
// Извлекает WebSocket ключ из HTTP запроса handshake

```

```

private string ExtractWebSocketKey(string httpRequest)
{
    string search = "Sec-WebSocket-Key: ";
    int start = httpRequest.IndexOf(search);
    if (start == -1) return null;
    start += search.Length;
    int end = httpRequest.IndexOf("\r\n", start);
    return httpRequest.Substring(start, end - start).Trim();
}
// Вычисляет Асепт ключ для WebSocket handshake
private string ComputeWebSocketAccept(string key)
{
    const string magic = "258EAF5E914-47DA-95CA-C5AB0DC85B11";
    using var sha1 = SHA1.Create();
    byte[] hash = sha1.ComputeHash(Encoding.UTF8.GetBytes(key + magic));
    return Convert.ToBase64String(hash);
}
// Отправляет HTTP ответ с указанным кодом
private async Task SendHttpResponse(NetworkStream stream, int code, string message)
{
    string response = $"HTTP/1.1 {code} {message}\r\nContent-Length: 0\r\n\r\n";
    await stream.WriteAsync(Encoding.ASCII.GetBytes(response));
}
// Обрабатывает входящее сообщение, маршрутизирует по типу команды
private async Task ProcessMessage(string connectionId, string message)
{
    if (string.IsNullOrEmpty(message) || message.Length == 0) return;
    try
    {
        if (message.Contains("\"type\":\"ping\""))
        {
            await SendToClient(connectionId, new { type = "pong" });
            return;
        }
        var options = new JsonSerializerOptions { PropertyNameCaseInsensitive = true };
        var command = JsonSerializer.Deserialize<WebSocketCommand>(message, options);
        if (command == null) return;
        Console.WriteLine($"[RawWebSocket] Команда от {connectionId}: {command.Type}");
        switch (command.Type)
        {
            case "register": break;
            case "create_room": await CreateRoom(connectionId, command); break;
            case "restart_game": await RestartGame(connectionId, command); break;
            case "join_room": await JoinRoom(connectionId, command); break;
            case "start_game": await StartGame(connectionId, command); break;
            case "flip_card": await FlipCard(connectionId, command); break;
            case "next_turn": await NextTurn(connectionId, command); break;
            case "force_update": await ForceUpdate(connectionId, command); break;
            default:
                Console.WriteLine($"[RawWebSocket] Неизвестная команда: {command.Type}");
        }
    }
}

```

```

        break;
    }
}
catch (Exception ex)
{
    Console.WriteLine($"[RawWebSocket] Ошибка обработки сообщения:
{ex.Message}\nСообщение: {message}");
    try { await SendToClient(connectionId, new { type = "error", message = ex.Message
}); } catch { }
}
}
// Создает новую игровую комнату
private async Task CreateRoom(string connectionId, WebSocketCommand command)
{
    int boardSize = command.BoardSize;
    if (boardSize != 4 && boardSize != 6 && boardSize != 8) boardSize = 4;
    var room = _roomManager.CreateRoom(command.PlayerName, connectionId,
boardSize);
    _connectionToRoom[connectionId] = room.RoomCode;
    await SendToClient(connectionId, new
    {
        type = "room_created",
        roomCode = room.RoomCode,
        boardSize = room.BoardSize
    });
    await UpdatePlayersList(room.RoomCode);
}
// Перезапускает игру в комнате (только для создателя)
private async Task RestartGame(string connectionId, WebSocketCommand command)
{
    var room = _roomManager.GetRoom(command.RoomCode);
    if (room == null) return;
    var creator = room.Players.FirstOrDefault();
    if (creator?.ConnectionId != connectionId) return;
    _roomManager.RestartGame(room);
    await SendToRoom(command.RoomCode, new
    {
        type = "game_restarted",
        boardSize = room.BoardSize
    });
    await UpdatePlayersList(command.RoomCode);
}
// Добавляет игрока в существующую комнату
private async Task JoinRoom(string connectionId, WebSocketCommand command)
{
    var room = _roomManager.GetRoom(command.RoomCode);
    if (room == null)
    {
        await SendToClient(connectionId, new { type = "error", message = "Комната не
найдена" });
        return;
    }
}

```

```

        if (room.GameStarted)
        {
            await SendToClient(connectionId, new { type = "error", message = "Игра уже
началась" });
            return;
        }
        if (room.Players.Count >= 4)
        {
            await SendToClient(connectionId, new { type = "error", message = "Комната
заполнена" });
            return;
        }
        // Проверяем, не присоединился ли уже этот connectionId
        if (room.Players.Any(p => p.ConnectionId == connectionId))
        {
            Console.WriteLine($"[RawWebSocket] Клиент {connectionId} уже в комнате
{command.RoomCode}");
            await SendToClient(connectionId, new { type = "board_size_info", boardSize =
room.BoardSize });
            await UpdatePlayersList(command.RoomCode);
            return;
        }
        var result = _roomManager.JoinRoom(command.RoomCode, command.PlayerName,
connectionId);
        if (result == null)
        {
            await SendToClient(connectionId, new { type = "error", message = "Не удалось
присоединиться" });
            return;
        }
        _connectionToRoom[connectionId] = command.RoomCode;
        Console.WriteLine($"[RawWebSocket] Игрок {command.PlayerName}
({connectionId}) присоединился к комнате {command.RoomCode}");
        await SendToClient(connectionId, new { type = "board_size_info", boardSize =
room.BoardSize });
        await UpdatePlayersList(command.RoomCode);
        if (room.Players.Count >= 2 && !room.GameStarted)
        {
            await SendToRoom(command.RoomCode, new { type = "ready_to_start" });
        }
    }
    // Запускает игру (только для создателя комнаты)
    private async Task StartGame(string connectionId, WebSocketCommand command)
    {
        var room = _roomManager.GetRoom(command.RoomCode);
        if (room == null || room.GameStarted) return;
        var creator = room.Players.FirstOrDefault();
        if (creator?.ConnectionId != connectionId) return;
        _roomManager.InitializeBoard(room);
        room.GameStarted = true;
        room.CurrentPlayerIndex = 0;
        room.PendingFirstCardIndex = null;
    }

```

```

room.IsProcessing = false;
var boardData = room.Board.Select(c => new
{
    c.Id,
    c.PairId,
    c.IsFlipped,
    c.IsMatched,
    c.ImagePath
}).ToList();
var currentPlayer = _roomManager.GetCurrentPlayer(room);
await SendToRoom(command.RoomCode, new
{
    type = "game_started",
    boardData = boardData,
    firstPlayerId = currentPlayer?.ConnectionId,
    boardSize = room.BoardSize
});
await UpdatePlayersList(command.RoomCode);
}
// Обрабатывает переворот карточки игроком
private async Task FlipCard(string connectionId, WebSocketCommand command)
{
    var room = _roomManager.GetRoom(command.RoomCode);
    if (room == null || !room.GameStarted || room.GameOver || room.IsProcessing) return;
    var currentPlayer = _roomManager.GetCurrentPlayer(room);
    if (currentPlayer?.ConnectionId != connectionId) return;
    if (command.CardIndex < 0 || command.CardIndex >= room.Board.Count) return;
    var card = room.Board[command.CardIndex];
    if (card.IsFlipped || card.IsMatched) return;
    if (room.PendingFirstCardIndex == null)
    {
        // Первая карта в ходе
        card.IsFlipped = true;
        room.PendingFirstCardIndex = command.CardIndex;
        await SendToRoom(command.RoomCode, new
        {
            type = "card_flipped",
            cardIndex = command.CardIndex,
            imagePath = card.ImagePath
        });
    }
    else
    {
        // Вторая карта в ходе - проверяем совпадение
        int firstIndex = room.PendingFirstCardIndex.Value;
        var firstCard = room.Board[firstIndex];
        card.IsFlipped = true;
        await SendToRoom(command.RoomCode, new
        {
            type = "card_flipped",
            cardIndex = command.CardIndex,
            imagePath = card.ImagePath });
    }
}

```

```

bool isMatch = (firstCard.PairId == card.PairId);
room.PendingFirstCardIndex = null;
if (isMatch)
{
    firstCard.IsMatched = true;
    card.IsMatched = true;
    currentPlayer.Score++;
    await SendToRoom(command.RoomCode, new
    {
        type = "match_found",
        firstCardIndex = firstIndex,
        secondCardIndex = command.CardIndex,
        playerName = currentPlayer.Name,
        score = currentPlayer.Score
    });
    await UpdatePlayersList(command.RoomCode);
    if (_roomManager.CheckGameOver(room))
    {
        await SendToRoom(command.RoomCode, new { type = "game_over", winnerId
= room.WinnerId });
    }
    else
    {
        await SendToRoom(command.RoomCode, new { type = "turn_changed",
nextPlayerId = currentPlayer.ConnectionId });
    }
}
else
{
    room.IsProcessing = true;
    await SendToRoom(command.RoomCode, new
    {
        type = "no_match",
        firstCardIndex = firstIndex,
        secondCardIndex = command.CardIndex
    });
}
}
}
// Переключает ход на следующего игрока
private async Task NextTurn(string connectionId, WebSocketCommand command)
{
    var room = _roomManager.GetRoom(command.RoomCode);
    if (room == null) return;
    var currentPlayer = _roomManager.GetCurrentPlayer(room);
    if (currentPlayer?.ConnectionId != connectionId) return;
    if (room.GameOver) return;
    if (!room.IsProcessing) return;
    // Закрываем неподошедшие карты
    var flippedCards = room.Board
        .Select((card, idx) => new { card, idx })
        .Where(x => x.card.IsFlipped && !x.card.IsMatched)

```

```

        .ToList();
        foreach (var item in flippedCards) item.card.IsFlipped = false;
        room.PendingFirstCardIndex = null;
        _roomManager.NextPlayer(room);
        room.IsProcessing = false;
        var nextPlayer = _roomManager.GetCurrentPlayer(room);
        await UpdatePlayersList(command.RoomCode);
        await SendToRoom(command.RoomCode, new { type = "turn_changed", nextPlayerId =
nextPlayer?.ConnectionId });
    }
    // Принудительно обновляет состояние клиента
    private async Task ForceUpdate(string connectionId, WebSocketCommand command)
    {
        await UpdatePlayersList(command.RoomCode);
        var room = _roomManager.GetRoom(command.RoomCode);
        if (room != null)
        {
            await SendToClient(connectionId, new { type = "board_size_info", boardSize =
room.BoardSize });
            if (room.GameStarted && !room.GameOver)
            {
                var boardData = room.Board.Select(c => new
                {
                    c.Id,
                    c.PairId,
                    c.IsFlipped,
                    c.IsMatched,
                    c.ImagePath
                }).ToList();
                await SendToClient(connectionId, new
                {
                    type = "game_state",
                    boardData = boardData,
                    currentPlayerId = _roomManager.GetCurrentPlayer(room)?.ConnectionId,
                    boardSize = room.BoardSize
                });
            }
        }
    }
    // Обновляет список игроков для всех в комнате
    private async Task UpdatePlayersList(string roomCode)
    {
        var room = _roomManager.GetRoom(roomCode);
        if (room == null) return;
        var currentPlayerId = room.CurrentPlayerIndex < room.Players.Count
            ? room.Players[room.CurrentPlayerIndex].ConnectionId
            : null;
        var playersInfo = room.Players.Select(p => new
        {
            connectionId = p.ConnectionId,
            name = p.Name,
            score = p.Score,

```

```

        isCurrent = p.ConnectionId == currentPlayerId,
        isOwner = p.IsOwner
    }).ToList();

    await SendToRoom(roomCode, new { type = "players_updated", players = playersInfo
});
}
// Обработывает отключение клиента: удаляет из комнаты, переключает ход
private async Task HandleDisconnect(string connectionId)
{
    Console.WriteLine($"[RawWebSocket] Клиент отключен: {connectionId}");
    _connectionToRoom.TryRemove(connectionId, out string roomCode);
    if (roomCode != null)
    {
        var room = _roomManager.GetRoom(roomCode);
        if (room != null)
        {
            var player = room.Players.FirstOrDefault(p => p.ConnectionId == connectionId);
            if (player != null)
            {
                string playerName = player.Name;
                bool wasCurrentPlayer = _roomManager.RemovePlayer(room, connectionId);
                await SendToRoom(roomCode, new { type = "player_left", playerName =
playerName });
                await UpdatePlayersList(roomCode);
                if (wasCurrentPlayer && room.GameStarted && !room.GameOver &&
room.Players.Count > 0)
                {
                    var nextPlayer = _roomManager.GetCurrentPlayer(room);
                    await SendToRoom(roomCode, new
                    {
                        type = "turn_changed",
                        nextPlayerId = nextPlayer?.ConnectionId
                    });
                }
            }
            if (room.Players.Count == 0)
                _roomManager.RemoveRoom(roomCode);
        }
    }
}
// Отправляет сообщение конкретному клиенту (потокбезопасно)
private async Task SendToClient(string connectionId, object data)
{
    if (!_clients.TryGetValue(connectionId, out var client)) return;
    if (!_clientLocks.TryGetValue(connectionId, out var semaphore)) return;
    if (!client.Connected) return;
    await semaphore.WaitAsync();
    try
    {
        var stream = client.GetStream();
        var json = JsonSerializer.Serialize(data);
    }
}

```

```

        await SendWebSocketFrame(stream, json);
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[RawWebSocket] Ошибка отправки клиенту {connectionId}:
{ex.Message}");
    }
    finally
    {
        semaphore.Release();
    }
}
// Отправляет сообщение всем игрокам в комнате
private async Task SendToRoom(string roomCode, object data)
{
    var room = _roomManager.GetRoom(roomCode);
    if (room == null) return;
    // Создаем копию списка игроков для безопасной итерации
    List<string> playerIds;
    lock (room)
    {
        playerIds = room.Players.Select(p => p.ConnectionId).ToList();
    }

    var tasks = playerIds.Select(id => SendToClient(id, data));
    await Task.WhenAll(tasks);
}
}
}
let websocket = null;
let currentRoomCode = null;
let boardSize = 4;
let cards = [];
let isMyTurn = false;
let waitingForFlipReset = false;
let playersList = [];
let reconnectAttempts = 0;
let heartbeatInterval = null;
let myConnectionId = null;
let amIOwner = false;
let gameStartedFlag = false;
// DOM элементы
const lobbyScreen = document.getElementById('lobbyScreen');
const waitingScreen = document.getElementById('waitingScreen');
const gameScreen = document.getElementById('gameScreen');
// Инициализация при загрузке страницы: генерирует или загружает connectionId и
подключается к WebSocket
document.addEventListener('DOMContentLoaded', () => {
    myConnectionId = localStorage.getItem('memory_connection_id');
    if (!myConnectionId) {
        myConnectionId = 'player_' + Math.random().toString(36).substring(2, 15);
        localStorage.setItem('memory_connection_id', myConnectionId);
    }
}

```

```

    }
    console.log("Мой ConnectionId:", myConnectionId);
    connectWebSocket();
    setupEventListeners();
  });
  // Устанавливает WebSocket соединение с сервером и настраивает обработчики событий
  function connectWebSocket() {
    const hostname = window.location.hostname;
    const wsUrl = `ws://${hostname}:8081`;
    console.log("Подключение к WebSocket (ручной):", wsUrl);
    websocket = new WebSocket(wsUrl);
    websocket.onopen = () => {
      console.log("WebSocket подключен");
      reconnectAttempts = 0;
      // Регистрируем клиента на сервере
      websocket.send(JSON.stringify({ type: "register", connectionId: myConnectionId }));
      if (heartbeatInterval) clearInterval(heartbeatInterval);
      heartbeatInterval = setInterval(() => {
        if (websocket?.readyState === WebSocket.OPEN) {
          websocket.send(JSON.stringify({ type: "ping", connectionId: myConnectionId }));
        }
      }, 30000);
    };
    websocket.onmessage = (event) => {
      const message = JSON.parse(event.data);
      console.log("Получено:", message);
      handleMessage(message);
    };
    websocket.onclose = () => {
      console.log("WebSocket отключен");
      if (heartbeatInterval) clearInterval(heartbeatInterval);
      if (reconnectAttempts < 5) {
        reconnectAttempts++;
        setTimeout(connectWebSocket, 2000 * reconnectAttempts);
      }
    };
    websocket.onerror = (error) => {
      console.error("WebSocket ошибка:", error);
    };
  }
  // Обработывает входящие сообщения от WebSocket сервера
  function handleMessage(message) {
    switch (message.type) {
      case "connected":
        console.log("✅ Сервер подтвердил соединение");
        break;
      case "room_created":
        currentRoomCode = message.roomCode;
        boardSize = message.boardSize;
        document.getElementById('displayRoomCode').innerText = message.roomCode;
        document.getElementById('gameRoomCode').innerText = message.roomCode;
        document.getElementById('waitingBoardSize').innerText = message.boardSize;
    }
  }

```

```

    document.getElementById('waitingBoardSize2').innerText = message.boardSize;
    document.getElementById('startGameBtn').style.display = 'block';
    showScreen('waitingScreen');
    break;
case "game_restarted":
    gameStartedFlag = false;
    console.log("Игра перезапущена!");
    document.getElementById('gameOverModal').style.display = 'none';
    waitingForFlipReset = false;
    isMyTurn = false;
    sendCommand("force_update", { roomCode: currentRoomCode });
    showScreen('waitingScreen');
    document.getElementById('startGameBtn').style.display = 'block';
    const restartInfoText = document.getElementById('waitingInfoText');
    restartInfoText.textContent = 'Игра перезапущена! Нажмите "Начать игру" для новой
игры';
    restartInfoText.classList.add('ready');
    break;
case "game_state":
    cards = message.boardData;
    boardSize = message.boardSize;
    isMyTurn = (message.currentPlayerId === myConnectionId);
    waitingForFlipReset = false;
    renderGameBoard();
    updateTurnIndicator();
    updatePlayersList(playersList);
    break;
case "board_size_info":
    boardSize = message.boardSize;
    document.getElementById('waitingBoardSize').innerText = message.boardSize;
    document.getElementById('waitingBoardSize2').innerText = message.boardSize;
    break;
case "ready_to_start":
    const readyInfoText = document.getElementById('waitingInfoText');
    readyInfoText.textContent = '☑ Игроки собраны! Нажмите "Начать игру"';
    readyInfoText.classList.add('ready');
    break;
case "game_started":
    console.log("Игра началась! firstPlayerId:", message.firstPlayerId);
    cards = message.boardData;
    boardSize = message.boardSize;
    isMyTurn = (message.firstPlayerId === myConnectionId);
    waitingForFlipReset = false;
    renderGameBoard();
    updateTurnIndicator();
    showScreen('gameScreen');
    showTemporaryMessage(isMyTurn ? "Игра началась! Ваш ход!" : "Игра началась!
Ход соперника...", 'info');
    break;
case "card_flipped":
    if (cards[message.cardIndex]) {
        cards[message.cardIndex].isFlipped = true;

```

```

        if (message.imagePath) cards[message.cardIndex].imagePath = message.imagePath;
        renderGameBoard();
    }
    break;
case "players_updated":
    console.log("Список игроков:", message.players);
    playersList = message.players;
    const me = playersList.find(p => p.connectionId === myConnectionId);
    amIOwner = me ? me.isOwner : false;
    updatePlayersList(playersList);
    updateOwnerButtons();
    break;
case "match_found":
    if (cards[message.firstCardIndex]) cards[message.firstCardIndex].isMatched = true;
    if (cards[message.secondCardIndex]) cards[message.secondCardIndex].isMatched = true;
    renderGameBoard();
    showTemporaryMessage(` ${message.playerName} нашёл пару!`, 'success');
    break;
case "no_match":
    waitingForFlipReset = true;
    setTimeout(() => {
        if (cards[message.firstCardIndex]) cards[message.firstCardIndex].isFlipped = false;
        if (cards[message.secondCardIndex]) cards[message.secondCardIndex].isFlipped =
false;
        renderGameBoard();
        if (isMyTurn && currentRoomCode) {
            sendCommand("next_turn", { roomCode: currentRoomCode });
        } else {
            waitingForFlipReset = false;
        }
    }, 1200);
    break;
case "turn_changed":
    const oldIsMyTurn = isMyTurn;
    isMyTurn = (message.nextPlayerId === myConnectionId);
    if (isMyTurn) {
        waitingForFlipReset = false;
        if (!oldIsMyTurn) showTemporaryMessage("Ваш ход!", 'success');
    } else {
        waitingForFlipReset = true;
    }
    updateTurnIndicator();
    renderGameBoard();
    break;
case "game_over":
    gameStartedFlag = false;
    const modalText = document.getElementById('winnerMessage');
    if (message.winnerId === "DRAW") {
        modalText.innerText = "Ничья!";
    } else if (message.winnerId === myConnectionId) {
        modalText.innerText = "Поздравляем! Вы победили!";
    } else {

```

```

        const winner = playersList.find(p => p.connectionId === message.winnerId);
        modalText.innerText = `Победил: ${winner ? winner.name : "Оппонент"}`;
    }
    document.getElementById('gameOverModal').style.display = 'flex';
    break;
case "player_left":
    showTemporaryMessage(`${message.playerName} покинул игру`, 'warning');
    break;
case "error":
    alert(message.message);
    if (message.message.includes("не найдена")) location.reload();
    break;
}
}
// Отправляет команду на сервер через WebSocket
function sendCommand(type, data) {
    if (websocket?.readyState !== WebSocket.OPEN) {
        console.error("WebSocket не подключен");
        return;
    }
    const command = { type, connectionId: myConnectionId, ...data };
    console.log("Отправка:", command);
    websocket.send(JSON.stringify(command));
}
// Настраивает обработчики событий для кнопок интерфейса
function setupEventListeners() {
    document.getElementById('createRoomBtn').addEventListener('click', createRoom);
    document.getElementById('playAgainBtn').addEventListener('click', playAgain);
    document.getElementById('joinRoomBtn').addEventListener('click', joinRoom);
    document.getElementById('startGameBtn').addEventListener('click', startGame);
    document.getElementById('leaveWaitingBtn').addEventListener('click', () =>
location.reload());
    document.getElementById('leaveGameBtn').addEventListener('click', () => location.reload());
    document.getElementById('closeModalBtn').addEventListener('click', () => {
        document.getElementById('gameOverModal').style.display = 'none';
        location.reload();
    });
}
// Создает новую игровую комнату
function createRoom() {
    const playerName = document.getElementById('createPlayerName').value.trim();
    const boardSizeValue = parseInt(document.getElementById('boardSize').value);
    if (!playerName) { alert('Введите имя'); return; }
    if (playerName.length < 2) { alert('Имя минимум 2 символа'); return; }
    sendCommand("create_room", { playerName: playerName, boardSize: boardSizeValue });
}
// Запрашивает перезапуск игры (только для владельца комнаты)
function playAgain() {
    if (currentRoomCode) {
        document.getElementById('gameOverModal').style.display = 'none';
        sendCommand("restart_game", { roomCode: currentRoomCode });
        showTemporaryMessage("Перезапуск игры...", 'info') }}

```

```

// Присоединяется к существующей комнате по коду
function joinRoom() {
    const playerName = document.getElementById('joinPlayerName').value.trim();
    const roomCode = document.getElementById('roomCode').value.trim().toUpperCase();
    if (!playerName) { alert('Введите имя'); return; }
    if (!roomCode) { alert('Введите код комнаты'); return; }
    if (roomCode.length !== 4) { alert('Код из 4 символов'); return; }
    currentRoomCode = roomCode;
    document.getElementById('displayRoomCode').innerText = roomCode;
    document.getElementById('gameRoomCode').innerText = roomCode;
    document.getElementById('startGameBtn').style.display = 'none';
    showScreen('waitingScreen');
    sendCommand("join_room", { roomCode: roomCode, playerName: playerName });
}
// Запускает игру (только для владельца комнаты)
function startGame() {
    if (currentRoomCode) {
        document.getElementById('startGameBtn').style.display = 'none';
        sendCommand("start_game", { roomCode: currentRoomCode });
    }
}
// Переворачивает карточку на игровом поле
function flipCard(cardIndex) {
    if (!isMyTurn) { showTemporaryMessage("Не ваш ход!", 'warning'); return; }
    if (waitingForFlipReset) { showTemporaryMessage("Подождите...", 'info'); return; }
    const card = cards[cardIndex];
    if (!card || card.isFlipped || card.isMatched) return;
    sendCommand("flip_card", { roomCode: currentRoomCode, cardIndex: cardIndex });
}
// Обновляет отображение списка игроков в интерфейсе
function updatePlayersList(players) {
    console.log("=== updatePlayersList вызван ===");
    console.log("Данные игроков:", JSON.stringify(players));
    const waitingList = document.getElementById('waitingPlayersList');
    const scoresContainer = document.getElementById('playersScores');
    console.log("waitingList элемент:", waitingList);
    console.log("scoresContainer элемент:", scoresContainer);
    // Обновляем экран ожидания
    if (waitingList) {
        waitingList.innerHTML = "";
        for (let i = 0; i < players.length; i++) {
            const p = players[i];
            const li = document.createElement('li');
            li.textContent = p.name + (p.isCurrent ? ' 🎯 : ' + (p.connectionId ===
myConnectionId ? ' (Вы) : ' : ' ');
            waitingList.appendChild(li);
            console.log("Создан LI с текстом:", li.textContent);
        }
    }
    // Обновляем игровой экран
    if (scoresContainer) {
        scoresContainer.innerHTML = "";
    }
}

```

```

    for (let i = 0; i < players.length; i++) {
        const p = players[i];
        const div = document.createElement('div');
        div.className = 'player-score-card';
        if (p.isCurrent) div.classList.add('current-turn');
        const h4 = document.createElement('h4');
        h4.textContent = p.name + (p.connectionId === myConnectionId ? ' (Вы)' : '');
        const scoreDiv = document.createElement('div');
        scoreDiv.className = 'score';
        scoreDiv.textContent = p.score !== undefined ? p.score : 0;
        div.appendChild(h4);
        div.appendChild(scoreDiv);
        scoresContainer.appendChild(div);
        console.log("Создана карточка:", h4.textContent, "счёт:", scoreDiv.textContent);
    }
}
}
// Отрисовывает игровое поле на основе текущего состояния карт
function renderGameBoard() {
    const board = document.getElementById('gameBoard');
    if (!board) return;
    board.setAttribute('data-size', boardSize);
    board.innerHTML = "";
    if (!cards || cards.length === 0) return;
    cards.forEach((card, index) => {
        const cardElement = document.createElement('div');
        cardElement.className = 'card';
        if (card.isMatched) {
            cardElement.classList.add('matched');
        } else if (card.isFlipped) {
            cardElement.classList.add('flipped');
            if (card.imagePath) {
                const img = document.createElement('img');
                img.src = card.imagePath;
                img.onerror = () => { cardElement.textContent = '?'; };
                cardElement.appendChild(img);
            } else {
                cardElement.textContent = '?';
            }
        } else {
            const backDiv = document.createElement('div');
            backDiv.style.cssText = 'background: linear-gradient(145deg, #667eea, #764ba2);
width:100%; height:100%; border-radius:12px; display:flex; align-items:center; justify-
content:center; font-size:1.5em; color:white;';
            backDiv.textContent = '?';
            cardElement.appendChild(backDiv);
        }
        if (!card.isMatched && !card.isFlipped && isMyTurn && !waitingForFlipReset) {
            cardElement.style.cursor = 'pointer';
            cardElement.onclick = (e) => { e.stopPropagation(); flipCard(index); };
        }
        board.appendChild(cardElement);
    });
}

```

```

// Обновляет индикатор текущего хода
function updateTurnIndicator() {
    const turnText = document.getElementById('turnText');
    const turnIndicator = document.getElementById('turnIndicator');
    if (isMyTurn) {
        turnText.textContent = '🔥 ВАШ ХОД!';
        turnIndicator.classList.add('your-turn');
    } else {
        turnText.textContent = '⌚ Ход соперника...';
        turnIndicator.classList.remove('your-turn');
    }
}

// Переключает видимость экранов (лобби/ожидание/игра)
function showScreen(screenId) {
    lobbyScreen.classList.remove('active');
    waitingScreen.classList.remove('active');
    gameScreen.classList.remove('active');
    document.getElementById(screenId).classList.add('active');
}

// Показывает временное всплывающее уведомление
function showTemporaryMessage(message, type) {
    let bgColor = type === 'success' ? '#48bb78' : (type === 'warning' ? '#ed8936' : '#667eea');
    const notification = document.createElement('div');
    notification.textContent = message;
    notification.style.cssText = `
        position: fixed;
        bottom: 30px;
        right: 30px;
        padding: 12px 24px;
        background: ${bgColor};
        color: white;
        border-radius: 12px;
        z-index: 2000;
        font-weight: bold;
        box-shadow: 0 4px 15px rgba(0,0,0,0.2);
    `;
    document.body.appendChild(notification);
    setTimeout(() => {
        notification.remove();
    }, 2500);
}

// Обновляет видимость кнопок владельца комнаты
function updateOwnerButtons() {
    const startGameBtn = document.getElementById('startGameBtn');
    const playAgainBtn = document.getElementById('playAgainBtn');
    if (startGameBtn) {
        startGameBtn.style.display = (amIOwner && currentRoomCode && !gameStartedFlag) ?
'block' : 'none';
    }
    if (playAgainBtn) {
        playAgainBtn.style.display = amIOwner ? 'inline-block' : 'none';
    }
}

```

```

// Экранирует HTML специальные символы для безопасности
function escapeHtml(str) {
    if (!str) return "";
    return str.replace(/[\&<>]/g, m => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;' }[m]));
}
using System.Collections.Concurrent;
using MemoryGame.Models;
namespace MemoryGame.Services
{
    public class GameRoomManager
    {
        private readonly ConcurrentDictionary<string, GameRoom> _rooms = new();
        private static readonly Random _random = new();
        private const string RoomCodeChars =
"ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
        private readonly string[] _imagePaths;
        public GameRoomManager()
        {
            _imagePaths = new string[32];
            for (int i = 0; i < 32; i++)
            {
                _imagePaths[i] = $"/images/card{i + 1}.jpg";
            }
        }
        // Создает новую игровую комнату с уникальным кодом
        public GameRoom CreateRoom(string playerName, string connectionId, int boardSize = 4)
        {
            if (boardSize != 4 && boardSize != 6 && boardSize != 8)
            {
                boardSize = 4;
            }
            string roomCode;
            do
            {
                roomCode = GenerateRoomCode();
            } while (_rooms.ContainsKey(roomCode));
            var room = new GameRoom
            {
                RoomCode = roomCode,
                GameStarted = false,
                GameOver = false,
                CurrentPlayerIndex = 0,
                PendingFirstCardIndex = null,
                BoardSize = boardSize
            };
            var player = new Player
            {
                ConnectionId = connectionId,
                Name = playerName,
                Score = 0,
                IsOwner = true
            };
        }
    }
}

```

```

        room.Players.Add(player);
        _rooms.TryAdd(roomCode, room);
        return room;
    }
    // Перезапускает игру: сбрасывает состояние и обнуляет очки
    public void RestartGame(GameRoom room)
    {
        room.GameStarted = false;
        room.GameOver = false;
        room.WinnerId = null;
        room.CurrentPlayerIndex = 0;
        room.PendingFirstCardIndex = null;
        room.IsProcessing = false;
        foreach (var player in room.Players)
        {
            player.Score = 0;
        }
        InitializeBoard(room);
    }
    // Добавляет игрока в существующую комнату (максимум 4 игрока)
    public GameRoom JoinRoom(string roomCode, string playerName, string connectionId)
    {
        if (!_rooms.TryGetValue(roomCode.ToUpper(), out var room))
            return null;
        if (room.GameStarted)
            return null;
        if (room.Players.Count >= 4)
            return null;
        var player = new Player
        {
            ConnectionId = connectionId,
            Name = playerName,
            Score = 0,
            IsOwner = false
        };
        room.Players.Add(player);
        return room;
    }
    // Возвращает комнату по коду
    public GameRoom GetRoom(string roomCode)
    {
        _rooms.TryGetValue(roomCode.ToUpper(), out var room);
        return room;
    }
    // Удаляет комнату из хранилища
    public bool RemoveRoom(string roomCode)
    {
        return _rooms.TryRemove(roomCode.ToUpper(), out _);
    }
    // Возвращает все активные комнаты
    public IEnumerable<GameRoom> GetAllRooms()
    {
        return _rooms.Values;
    }

```

```

// Генерирует случайный 4-символьный код комнаты
private string GenerateRoomCode()
{
    var code = new char[4];
    for (int i = 0; i < 4; i++)
    {
        code[i] = RoomCodeChars[_random.Next(RoomCodeChars.Length)];
    }
    return new string(code);
}
// Создает и перемешивает колоду карт для указанной комнаты
public void InitializeBoard(GameRoom room)
{
    var cards = new List<Card>();
    int totalCards = room.BoardSize * room.BoardSize;
    int pairCount = totalCards / 2;
    for (int pairId = 0; pairId < pairCount; pairId++)
    {
        string imagePath = _imagePaths[pairId % _imagePaths.Length];
        cards.Add(new Card
        {
            Id = pairId * 2,
            PairId = pairId,
            IsFlipped = false,
            IsMatched = false,
            ImagePath = imagePath
        });
        cards.Add(new Card
        {
            Id = pairId * 2 + 1,
            PairId = pairId,
            IsFlipped = false,
            IsMatched = false,
            ImagePath = imagePath
        });
    }
    // Алгоритм Фишера-Йетса для перемешивания
    for (int i = cards.Count - 1; i > 0; i--)
    {
        int j = _random.Next(i + 1);
        (cards[i], cards[j]) = (cards[j], cards[i]);
    }

    room.Board = cards;
}
// Возвращает текущего игрока в комнате
public Player GetCurrentPlayer(GameRoom room)
{
    if (room.Players.Count == 0) return null;
    return room.Players[room.CurrentPlayerIndex];
}

```

```

// Переключает ход на следующего игрока
public void NextPlayer(GameRoom room)
{
    if (room.Players.Count == 0) return;
    room.CurrentPlayerIndex = (room.CurrentPlayerIndex + 1) % room.Players.Count;
}
// Проверяет, закончена ли игра, и определяет победителя
public bool CheckGameOver(GameRoom room)
{
    bool allMatched = room.Board.All(c => c.IsMatched);
    if (allMatched)
    {
        room.GameOver = true;
        int maxScore = room.Players.Max(p => p.Score);
        var winners = room.Players.Where(p => p.Score == maxScore).ToList();
        if (winners.Count > 1)
        {
            room.WinnerId = "DRAW";
        }
        else
        {
            room.WinnerId = winners.First().ConnectionId;
        }
        return true;
    }
    return false;
}
// Удаляет игрока из комнаты, корректирует ход при необходимости
public bool RemovePlayer(GameRoom room, string connectionId)
{
    var playerIndex = room.Players.FindIndex(p => p.ConnectionId == connectionId);
    if (playerIndex == -1) return false;
    bool wasCurrentPlayer = (playerIndex == room.CurrentPlayerIndex);
    room.Players.RemoveAt(playerIndex);
    if (room.GameStarted && !room.GameOver && room.Players.Count > 0)
    {
        if (wasCurrentPlayer)
        {
            if (playerIndex <= room.CurrentPlayerIndex && room.CurrentPlayerIndex > 0)
            {
                room.CurrentPlayerIndex--;
            }
            if (room.CurrentPlayerIndex >= room.Players.Count)
            {
                room.CurrentPlayerIndex = 0;
            }
        }
        // Сбрасываем незавершенный ход при удалении текущего игрока
        if (room.IsProcessing)
        {
            var flippedCards = room.Board
                .Where(c => c.IsFlipped && !c.IsMatched)
                .ToList();
        }
    }
}

```

```

        foreach (var card in flippedCards)
        {
            card.IsFlipped = false;
        }
        room.PendingFirstCardIndex = null;
        room.IsProcessing = false;
    }
}
else
{
    if (playerIndex < room.CurrentPlayerIndex)
    {
        room.CurrentPlayerIndex--;
    }
}
}
return wasCurrentPlayer;
}
}
}

@{
    Layout = null;
    ViewData["Title"] = "Memory Game - Найди пару";
}
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0, user-scalable=no">
    <title>@ViewData["Title"]</title>
    <link rel="stylesheet" href="~/css/style.css">
</head>
<body>
    <div class="container">
        <!-- Экран входа в игру (лобби) -->
        <div id="lobbyScreen" class="screen active">
            <div class="lobby-container">
                <h1>🎮 Memory: Найди пару</h1>
                <p class="subtitle">Сетевая игра на память</p>

                <div class="game-modes">
                    <!-- Создание комнаты -->
                    <div class="mode-card">
                        <div class="mode-icon">🎲</div>
                        <h2>Создать комнату</h2>
                        <input type="text" id="createPlayerName" placeholder="Ваше имя"
maxlength="20" autocomplete="off">

                        <div class="board-size-selector">
                            <label>Размер поля:</label>

```

```

        <select id="boardSize">
            <option value="4">4×4 (8 пар, легко)</option>
            <option value="6">6×6 (18 пар, средне)</option>
            <option value="8">8×8 (32 пары, сложно)</option>
        </select>
    </div>

    <button id="createRoomBtn" class="btn-primary">+ Создать игру</button>
</div>

<!-- Присоединение к комнате -->
<div class="mode-card">
    <div class="mode-icon">🎯</div>
    <h2>Присоединиться</h2>
    <input type="text" id="joinPlayerName" placeholder="Ваше имя"
maxlength="20" autocomplete="off">
    <input type="text" id="roomCode" placeholder="Код комнаты" maxlength="4"
style="text-transform: uppercase" autocomplete="off">
    <button id="joinRoomBtn" class="btn-primary">🎯 Присоединиться</button>
</div>
</div>
</div>
<!-- Экран комнаты ожидания -->
<div id="waitingScreen" class="screen">
    <div class="waiting-container">
        <div class="waiting-header">
            <h1>🕒 Ожидание игроков</h1>
            <div class="room-code-display">
                <span class="label">Код комнаты:</span>
                <span class="code" id="displayRoomCode">----</span>
            </div>
        </div>
        <div class="board-size-info">
            📏 Размер поля: <span id="waitingBoardSize">4</span>×<span
id="waitingBoardSize2">4</span>
        </div>
        <div class="players-list">
            <h3>👥 Игроки в комнате:</h3>
            <ul id="waitingPlayersList"></ul>
        </div>
        <button id="startGameBtn" class="btn-success" style="display: none;">🚀 Начать
игру</button>
        <button id="leaveWaitingBtn" class="btn-secondary">🚪 Выйти из
комнаты</button>
        <p id="waitingInfoText" class="info-text">Ожидание подключения других игроков
(нужно минимум 2)...</p>
    </div>
</div>

```

```

<!-- Экран игры -->
<div id="gameScreen" class="screen">
  <div class="game-container">
    <div class="game-header">
      <div class="room-info">
        🏠 Комната: <span id="gameRoomCode">----</span>
      </div>
      <div class="turn-indicator" id="turnIndicator">
        <span id="turnText">Загрузка...</span>
      </div>
    </div>
    <div class="players-panel">
      <div id="playersScores"></div>
    </div>
    <div class="game-board" id="gameBoard">
      <!-- Игровое поле генерируется динамически -->
    </div>
    <div class="game-controls">
      <button id="leaveGameBtn" class="btn-secondary">🚪 Выйти из игры</button>
    </div>
  </div>
</div>
<!-- Модальное окно окончания игры -->
<div id="gameOverModal" class="modal">
  <div class="modal-content">
    <div class="modal-icon">🏆</div>
    <h2>Игра окончена!</h2>
    <p id="winnerMessage">Победитель: ...</p>
    <div style="display: flex; gap: 15px; justify-content: center;">
      <button id="playAgainBtn" class="btn-success">🔄 Сыграть еще раз</button>
      <button id="closeModalBtn" class="btn-primary">Закрыть</button>
    </div>
  </div>
</div>
</div>
<script src="~/js/memory-client.js"></script>
</body>
</html>

```

```

namespace MemoryGame.Models
{
    public class Player
    {
        public string ConnectionId { get; set; } //уникальный Id подключения
        public string Name { get; set; } //имя
        public int Score { get; set; } //счет
        public bool IsOwner { get; set; } //создатель комнаты
    }
}

```

```
namespace MemoryGame.Models
```

```
{  
    public class Card  
    {  
        public int Id { get; set; } //уникальный id  
        public int PairId { get; set; } //парный id  
        public bool IsFlipped { get; set; } //открыта ли карточка  
        public bool IsMatched { get; set; } //уже найдена пара  
        public string ImagePath { get; set; } //путь к картинке  
    }  
}
```

```
namespace MemoryGame.Models
```

```
{  
    public class GameRoom  
    {  
        public string RoomCode { get; set; } //код комнаты  
        public List<Player> Players { get; set; } = new(); //список игроков  
        public List<Card> Board { get; set; } = new(); //поле карточек  
        public int CurrentPlayerIndex { get; set; } //индекс игрока, чей ход  
        public bool GameStarted { get; set; } //начало игры  
        public bool GameOver { get; set; } //конец игры  
        public string WinnerId { get; set; } //id победителя  
        public int? PendingFirstCardIndex { get; set; } //индекс первой открытой карты  
        public int BoardSize { get; set; } = 4; //размер поля  
        public bool IsProcessing { get; set; } //обработка пары карт  
    }  
}
```

```
namespace MemoryGame.Models
```

```
{  
    public class WebSocketCommand  
    {  
        public string Type { get; set; } //тип команды  
        public string ConnectionId { get; set; } //id подключения клиента  
        public string RoomCode { get; set; } //код комнаты  
        public string PlayerName { get; set; } //имя игрока  
        public int BoardSize { get; set; } //размер игрового поля  
        public int CardIndex { get; set; } //индекс карточки  
        public int Score { get; set; } //счет игрока  
    }  
}
```

```
using MemoryGame.Services;  
using MemoryGame.Handlers;  
var builder = WebApplication.CreateBuilder(args);  
builder.Services.AddControllersWithViews();  
builder.Services.AddSingleton<GameRoomManager>();  
builder.Services.AddSingleton<RawWebSocketHandler>();  
var app = builder.Build();
```

```
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
}
app.UseStaticFiles();
app.UseRouting();
var wsHandler = app.Services.GetRequiredService<RawWebSocketHandler>();
_ = Task.Run(() => wsHandler.StartServer(8081));
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.Run();
```